When Decision is Possible

A Report for
MA4198 Mathematics Capstone Project
(Completed in AY25/26 Semester 2)

Under the supervision of
Prof. Tran Chieu Minh

Lee Yu Xuan Dallen
A0258289J

National University of Singapore
November 8, 2026

Contents

1 Making Things Precise **2**
1.1 Defining a Universal Language . . . 2
1.2 Terms, Formulas and Satisfiability . . . 3
1.3 Theories . . . 4
1.4 Definable Sets . . . 5
1.5 Proofs and Gödel's Completeness Theorem . . . 7
1.6 Homomorphisms, Embeddings and Isomorphisms . . . 8

2 Computability **11**
2.1 Turing Machines and the Church–Turing Thesis . . . 11
2.2 Computable Functions and Sets . . . 11
2.3 Gödel Numbering . . . 14

3 Quantifier Elimination **17**
3.1 Quantifier Elimination: Definition and Consequences . . . 17
3.2 General Strategy for Quantifier Elimination . . . 18
3.3 Quantifier Elimination for Presburger Arithmetic . . . 19
3.4 Quantifier Elimination for Real Closed Fields . . . 20

4 Decidability of Euclidean Geometry **22**
4.1 The Language of Euclidean Geometry . . . 22
4.2 Interpretations . . . 22
4.3 The Truth Lemma and Transfer of Decidability . . . 23
4.4 Interpreting Euclidean Geometry in RCF . . . 25

5 Quantifier Elimination for the theory of cyclic abelian groups **27**

This is my report on Model Theory for the course *MA4198 Mathematics Capstone Project* under Prof. Tran Chieu Minh.

In this report, we develop the mathematical logic foundations needed to understand Tarski's remarkable theorem on the decidability of Euclidean geometry. In Chapter 1, we rigorously define the language of first-order logic and the structures that give it meaning. In Chapter 2, we introduce a sufficient amount of computability theory in order to make precise the notion of decidability by means of an algorithm. We then introduce the heart of the report in Chapter 3, on quantifier elimination. We demonstrate quantifier elimination methods to two theories, one of them being the theory of real closed fields (by Tarski). In Chapter 4, we introduce the notion of interpretation to transfer decidability from real closed fields to Euclidean geometry. In the final chapter (Chapter 5), we prove quantifier elimination for the theory of cyclic abelian groups.

1 Making Things Precise

The goal of this chapter is to build a rigorous framework for mathematical reasoning from the ground up. We begin with the most primitive notion: a *language*, which is nothing more than a collection of symbols. We then give these symbols meaning via *structures*, and finally build up *formulas* that we can evaluate for truth or falsity inside a structure.

One would notice that there are many definitions to remember in this section. This is mainly due to the immense effort taken to define things we have taken for granted, such as a simple formula like $x = 0$.

1.1 Defining a Universal Language

Many definitions and proofs in this section are adapted from *An Invitation to Mathematical Logic* by Marker [1].

Definition 1.1 (Language [1, Definition 1.1, pg. 4])**.** A **language** $\mathcal{L}$ is given by specifying the following data:

1. A set of *function symbols* $\mathcal{F}$, and for each $f \in \mathcal{F}$, a positive integer n_f called the *arity* of f (so f is an n_f-ary function symbol).
2. A set of *relation symbols* $\mathcal{R}$, and for each $R \in \mathcal{R}$, a positive integer n_R called the *arity* of R.
3. A set of *constant symbols* $\mathcal{C}$.

Remark 1.2. Here are some remarks on languages.

1. Any of these three sets may be empty (See the example below of the language of order).
2. A language is purely syntactic: it is a formal alphabet. The meaning (or *interpretation*) will be supplied later when we define structures.

Example 1.3. The following are standard examples of languages.

1. $\mathcal{L}_{\text{ord}} = \{<\}$, the **language of order**. Here $<$ is a binary relation symbol ($n_< = 2$). There are no function symbols and no constants.
2. $\mathcal{L}_{\text{ring}} = \{0, 1, +, \cdot\}$, the **language of rings**, where:
   - $+$ and $\cdot$ are binary function symbols ($n_+ = n_\cdot = 2$),
   - 0 and 1 are constant symbols.
3. $\mathcal{L}_{\text{or}} = \{0, 1, +, \cdot, <\}$, the **language of ordered rings**. This extends $\mathcal{L}_{\text{ring}}$ by adding $<$ as a binary relation symbol.

Definition 1.4 (Structure [1, Definition 1.2, pg. 4])**.** An **$\mathcal{L}$-structure** $\mathcal{M}$ consists of:

1. A non-empty set M, called the *universe*, *domain*, or *underlying set* of $\mathcal{M}$.
2. For each $f \in \mathcal{F}$, a function $f^{\mathcal{M}} : M^{n_f} \to M$.
3. For each $R \in \mathcal{R}$, a set $R^{\mathcal{M}} \subseteq M^{n_R}$.
4. For each $c \in \mathcal{C}$, an element $c^{\mathcal{M}} \in M$.

We call $f^{\mathcal{M}}$, $R^{\mathcal{M}}$, and $c^{\mathcal{M}}$ the **interpretations** of the symbols f, R, and c in $\mathcal{M}$.

Example 1.5. We give structures corresponding to the languages in Example 1.3.

1. $(\mathbb{Z}, <)$ is an $\mathcal{L}_{\text{ord}}$-structure. Here the domain is $\mathbb{Z}$ and $<^{\mathcal{M}}$ is the usual integer ordering.
2. $(\mathbb{Z}, 0, 1, +, \cdot)$ is an $\mathcal{L}_{\text{ring}}$-structure: the domain is $\mathbb{Z}$, the constants $0^{\mathcal{M}} = 0$ and $1^{\mathcal{M}} = 1$, and $+^{\mathcal{M}}, \cdot^{\mathcal{M}}$ are the usual integer addition and multiplication.
3. $(\mathbb{R}, 0, 1, +, \cdot, <)$ is an $\mathcal{L}_{\text{or}}$-structure with the standard ordering on $\mathbb{R}$.

Remark 1.6. It is important to note that the *same language* can give rise to very different structures. For instance, $(\mathbb{Z}/2\mathbb{Z}, 0, 1, +, \cdot)$ and $(\mathbb{C}, 0, 1, +, \cdot)$ are both $\mathcal{L}_{\text{ring}}$-structures, even though they have completely different algebraic properties. The language is just the shared *vocabulary*; the structure assigns its *meaning*.

1.2 Terms, Formulas and Satisfiability

Now that we have languages and structures, we build the objects we want to reason about: *terms* (which denote elements of a structure) and *formulas* (which make claims about those elements).

Definition 1.7 (Set of $\mathcal{L}$-terms [1, Definition 1.3, pg. 5])**.** The set of **$\mathcal{L}$-terms** is the smallest set $\mathcal{T}$ such that:

1. $c \in \mathcal{T}$ for each constant symbol $c \in \mathcal{C}$.
2. Each variable symbol $v_i \in \mathcal{T}$, for $i = 1, 2, \ldots$
3. If $f \in \mathcal{F}$ is an n_f-ary function symbol and $t_1, \ldots, t_{n_f} \in \mathcal{T}$, then $f(t_1, \ldots, t_{n_f}) \in \mathcal{T}$.

Example 1.8. In $\mathcal{L}_{\text{ring}}$, the following are terms:

$$0, \quad 1, \quad v_1, \quad +(v_1, v_2), \quad \cdot(+(v_1, 1), v_2).$$

Definition 1.9 (Atomic formula [1, Definition 1.5, pg. 6])**.** An **atomic $\mathcal{L}$-formula** is an expression of one of the following forms:

1. $t_1 = t_2$, where $t_1, t_2 \in \mathcal{T}$ are terms.
2. $R(t_1, \ldots, t_{n_R})$, where $R \in \mathcal{R}$ and $t_1, \ldots, t_{n_R} \in \mathcal{T}$.

Definition 1.10 (Set of $\mathcal{L}$-formulas [1, Definition 1.5, pg. 6-7])**.** The set of **$\mathcal{L}$-formulas** is the smallest set $\mathcal{W}$ that contains all atomic $\mathcal{L}$-formulas and is closed under:

1. *Negation*: if $\phi \in \mathcal{W}$, then $\neg\phi \in \mathcal{W}$.
2. *Boolean connectives*: if $\phi, \psi \in \mathcal{W}$, then $(\phi \wedge \psi), (\phi \vee \psi), (\phi \to \psi) \in \mathcal{W}$.
3. *Quantification*: if $\phi \in \mathcal{W}$, then $\exists v_i\, \phi$ and $\forall v_i\, \phi$ are in $\mathcal{W}$, for any variable v_i.

Remark 1.11. The *complexity* of a formula is naturally measured by structural induction, that is, the number of logical connectives and quantifiers used to build it. This inductive structure is exploited constantly in proofs throughout this report (e.g., the Truth Lemma in Section 4).

Before we can evaluate whether a formula is true or false in a structure, we must handle variables. A formula like $v_1 + v_2 = 0$ is neither true nor false until we know what v_1 and v_2 refer to.

Definition 1.12 (Assignment [1, Definition 1.7, pg. 7-8])**.** Let $V = \{v_1, v_2, \ldots\}$ be the set of variable symbols and let $\mathcal{M}$ be an $\mathcal{L}$-structure with domain M. An *assignment* (or *variable assignment*) is a function $\sigma : V \to M$.

Given an assignment σ, every term t receives a value $t^{\mathcal{M}}[\sigma] \in M$, defined by:

- $c^{\mathcal{M}}[\sigma] = c^{\mathcal{M}}$ (the interpretation of the constant);
- $v_i^{\mathcal{M}}[\sigma] = \sigma(v_i)$;
- $f(t_1, \ldots, t_n)^{\mathcal{M}}[\sigma] = f^{\mathcal{M}}(t_1^{\mathcal{M}}[\sigma], \ldots, t_n^{\mathcal{M}}[\sigma])$.

We write $\sigma[v_i \mapsto a]$ for the assignment that agrees with σ on all variables except v_i, which it sends to $a \in M$.

Definition 1.13 (Satisfaction [1, Definition 1.9, pg. 9])**.** Let $\mathcal{M}$ be an $\mathcal{L}$-structure, $\sigma : V \to M$ an assignment, and ϕ an $\mathcal{L}$-formula. We define the *satisfaction relation* $\mathcal{M} \models \phi[\sigma]$ (read: "$\mathcal{M}$ satisfies ϕ under σ") by induction on the complexity of ϕ:

1. $\mathcal{M} \models (t_1 = t_2)[\sigma] \iff t_1^{\mathcal{M}}[\sigma] = t_2^{\mathcal{M}}[\sigma]$ in M.
2. $\mathcal{M} \models R(t_1, \ldots, t_n)[\sigma] \iff (t_1^{\mathcal{M}}[\sigma], \ldots, t_n^{\mathcal{M}}[\sigma]) \in R^{\mathcal{M}}$.
3. $\mathcal{M} \models (\neg\phi)[\sigma] \iff \mathcal{M} \not\models \phi[\sigma]$.
4. $\mathcal{M} \models (\phi \wedge \psi)[\sigma] \iff \mathcal{M} \models \phi[\sigma]$ and $\mathcal{M} \models \psi[\sigma]$.
5. $\mathcal{M} \models (\phi \vee \psi)[\sigma] \iff \mathcal{M} \models \phi[\sigma]$ or $\mathcal{M} \models \psi[\sigma]$.
6. $\mathcal{M} \models (\phi \to \psi)[\sigma] \iff \mathcal{M} \not\models \phi[\sigma]$ or $\mathcal{M} \models \psi[\sigma]$.
7. $\mathcal{M} \models (\exists v_i\, \phi)[\sigma] \iff \exists a \in M$ such that $\mathcal{M} \models \phi[\sigma[v_i \mapsto a]]$.
8. $\mathcal{M} \models (\forall v_i\, \phi)[\sigma] \iff \forall a \in M$, $\mathcal{M} \models \phi[\sigma[v_i \mapsto a]]$.

Example 1.14. Consider the $\mathcal{L}_{\text{ring}}$-structure $\mathcal{M} = (\mathbb{Z}, 0, 1, +, \cdot)$ and the formula $\phi \equiv \exists v_2\, (v_1 \cdot v_2 = 1)$.

Under an assignment with $\sigma(v_1) = 2$, this asks whether 2 is a unit in $\mathbb{Z}$. Since 2 has no multiplicative inverse in $\mathbb{Z}$, we have $\mathcal{M} \not\models \phi[\sigma]$. However, in $\mathcal{M}' = (\mathbb{Q}, 0, 1, +, \cdot)$, the same formula is satisfied, that is $\mathcal{M}' \models \phi$ because $2 \cdot \frac{1}{2} = 1$.

1.3 Theories

Definition 1.15 (Free and Bound Variables)**.** An occurrence of a variable v_i in a formula ϕ is *bound* if it falls within the scope of a quantifier $\exists v_i$ or $\forall v_i$, and *free* otherwise.

Definition 1.16 (Sentence)**.** An *$\mathcal{L}$-sentence* is an $\mathcal{L}$-formula with no free variables.

Sentences are the objects we can meaningfully call true or false in a structure, because their truth does not depend on any variable assignment.

Definition 1.17 (Truth in a Structure)**.** Let σ be any assignment and ϕ a sentence. Since ϕ has no free variables, the truth value of $\mathcal{M} \models \phi[\sigma]$ does not depend on σ. We therefore simply write $\mathcal{M} \models \phi$ and say that ϕ is *true* in $\mathcal{M}$, or that $\mathcal{M}$ is a *model* of ϕ.

Example 1.18.

1. The sentence $\forall v_1\, \forall v_2\, (v_1 \cdot v_2 = v_2 \cdot v_1)$ expresses commutativity of multiplication. It is true in $(\mathbb{Z}, 0, 1, +, \cdot)$ and $(\mathbb{R}, 0, 1, +, \cdot)$, but false in any structure whose multiplication is non-commutative.
2. The sentence $\exists v_1\, (v_1 \cdot v_1 = 1 + 1)$ asks whether $\sqrt{2}$ exists. It is true in $(\mathbb{R}, 0, 1, +, \cdot)$ but false in $(\mathbb{Q}, 0, 1, +, \cdot)$.

Definition 1.19 (Theory)**.** An *$\mathcal{L}$-theory* T is a set of $\mathcal{L}$-sentences. We say that a structure $\mathcal{M}$ is a *model* of T, written $\mathcal{M} \models T$, if $\mathcal{M} \models \phi$ for every $\phi \in T$.

Definition 1.20 (Completeness of a theory)**.** An $\mathcal{L}$-theory T is *complete* if for every $\mathcal{L}$-sentence ϕ, either $\phi \in T$ or $\neg\phi \in T$.

Given a structure $\mathcal{M}$, the *complete theory* of $\mathcal{M}$ is

$$\mathrm{Th}(\mathcal{M}) = \{\phi : \phi \text{ is an } \mathcal{L}\text{-sentence and } \mathcal{M} \models \phi\}.$$

Remark 1.21. Note that $\mathrm{Th}(\mathcal{M})$ is always a complete theory. The study of what properties are captured by $\mathrm{Th}(\mathcal{M})$ and whether $\mathrm{Th}(\mathcal{M})$ is decidable is the central theme of this report.

Proposition 1.22. *Let T be a complete $\mathcal{L}$-theory and $\mathcal{M}$ be an $\mathcal{L}$-structure. If $\mathcal{M} \models T$, then $T = \mathrm{Th}(\mathcal{M})$.*

Proof. We show equality by proving containment in both directions.

($\subseteq$) Let $\sigma \in T$. By the definition of a model, since $\mathcal{M} \models T$, then $\mathcal{M} \models \sigma$. By the definition of the complete theory of a structure, $\sigma \in \mathrm{Th}(\mathcal{M})$. Thus $T \subseteq \mathrm{Th}(\mathcal{M})$.

($\supseteq$) Let $\sigma \in \mathrm{Th}(\mathcal{M})$, so by definiton, $\mathcal{M} \models \sigma$. Suppose for sake of contradiction that $\sigma \notin T$. Then, since T is a complete theory, it must satisfy the condition that for any sentence ϕ, either $\phi \in T$ or $\neg\phi \in T$. Therefore, $\sigma \notin T \implies \neg\sigma \in T$.

Since $\mathcal{M} \models T$, it follows that $\mathcal{M} \models \neg\sigma$. This contradicts our initial assumption that $\mathcal{M} \models \sigma$, as a structure cannot satisfy both a sentence and its negation. Thus, σ must be in T, and $\mathrm{Th}(\mathcal{M}) \subseteq T$.

□

Example 1.23 (Examples of theories)**.**

1. The *theory of real closed fields* (RCF) consists of sentences axiomatizing the first-order properties of $(\mathbb{R}, 0, 1, +, \cdot, <)$. It is an ordered field in which every positive element has a square root and every odd-degree polynomial has a root.

2. *Presburger arithmetic* is the $\mathcal{L}_{\mathrm{Pres}}$-theory of $(\mathbb{Z}, +, 0, 1, \equiv_2, \equiv_3, \ldots)$, axiomatized by the abelian group axioms, characteristic zero, and the definition of congruences modulo n for each $n \geq 2$.

In chapter 3, we introduce a powerful methodology called quantifier elimination, where we show that each sentence in a theory is equivalent to a quantifier-free one. We will be showing that these two particular theories does in fact admit quantifier elimination.

1.4 Definable Sets

Definition 1.24 (Definable Set [1, Definition 1.51, pg. 24])**.** Let $\mathcal{M}$ be an $\mathcal{L}$-structure with domain M.

A subset $S \subseteq M^n$ is *definable* in $\mathcal{M}$ if there exists an $\mathcal{L}$-formula $\phi(v_1, \ldots, v_n)$ that acts as a membership test, that is, a tuple $(a_1, \ldots, a_n)$ belongs to S if and only if it satisfies ϕ in $\mathcal{M}$. Formally,

$$S \;=\; \phi(\mathcal{M}) \;=\; \big\{(a_1, \ldots, a_n) \in M^n : \mathcal{M} \models \phi(a_1, \ldots, a_n)\big\}.$$

Definability with parameters. Here, the formula ϕ uses only the symbols of $\mathcal{L}$. So, no specific elements of M are mentioned. However, the language $\mathcal{L}$ often cannot refer to specific elements of M directly. For instance, in $\mathcal{L}_{\mathrm{or}} = \{0, 1, +, \cdot, <\}$, the only named constants are 0

and 1; there is no symbol for 3 or π. So a set like $(3, \infty) \subseteq \mathbb{R}$ cannot be defined by any formula in $\mathcal{L}_{\text{or}}$ alone. This is undesirable because it limits the definable sets that we can construct.

To handle this, we allow ourselves to *borrow* finitely many elements $a_1, \ldots, a_k \in M$ from the structure itself, that we will call *parameters*, and treat them as additional constants inside the formula. Concretely, we take a formula $\phi(v_1, \ldots, v_n, w_1, \ldots, w_k)$ with two groups of free variables:

- $v_1, \ldots, v_n$: the *varying* variables, which range freely over M and determine membership in S.
- $w_1, \ldots, w_k$: the *parameter* variables, which are permanently fixed to the chosen elements $a_1, \ldots, a_k \in M$.

We say $S \subseteq M^n$ is *definable with parameters* $a_1, \ldots, a_k$ if

$$S \;=\; \big\{(b_1, \ldots, b_n) \in M^n : \mathcal{M} \models \phi(b_1, \ldots, b_n, a_1, \ldots, a_k)\big\}$$

The parameters $a_1, \ldots, a_k$ are fixed throughout; only $b_1, \ldots, b_n$ vary. Intuitively, the parameters allow the formula to "refer to" specific elements of M that the language $\mathcal{L}$ cannot name on its own.

Example 1.25. Work in the structure $\mathcal{M} = (\mathbb{R}, 0, 1, +, \cdot, <)$.

1. We want $S = (0, \infty)$. The formula

   $$\phi(v_1) \;\equiv\; \exists v_2\,(v_2 \cdot v_2 = v_1) \;\wedge\; v_1 > 0$$

   is a perfect membership test: $x \in \mathbb{R}$ satisfies ϕ if and only if x has a square root and is positive, which in $\mathbb{R}$ means precisely that $x > 0$. So

   $$\phi(\mathbb{R}) = (0, \infty) = S$$

2. We want $S = (3, \infty)$. No parameter-free formula works, because $\mathcal{L}$ has no symbol for the specific constant 3. But if we allow the *parameter* $a = 3$, then the formula $\psi(v_1, w) \equiv w < v_1$ with w fixed to 3 gives a perfect membership test: x satisfies $\psi(x, 3)$ if and only if $3 < x$. So

   $$\psi(\mathbb{R}, 3) = (3, \infty) = S$$

Definition 1.26 (Logical Consequence)**.** Let T be a theory and ϕ a sentence. We say ϕ is a *logical consequence* of T, written $T \models \phi$, if every model of T also satisfies ϕ. In other words, there is no model in which all of T is true but ϕ is false.

Example 1.27. Let T be the theory of groups, and let ϕ be the sentence $\forall x\,(x \cdot e = x)$ (the identity element is a right identity). Since every group satisfies this by definition, ϕ holds in every model of T, so $T \models \phi$.

On the other hand, the sentence $\forall x \forall y\,(x \cdot y = y \cdot x)$ (commutativity) is *not* a logical consequence of T: the symmetric group S_3 is a model of T in which commutativity fails.

Remark 1.28.

1. The notion of a logical consequence is exactly what a mathematician would think it means. Precisely, it means that if ϕ is a logical consequence of a theory T (which we recall is a set of $\mathcal{L}$-sentences), then every model of T must be able to determine ϕ to be true.
2. This definition is purely *semantic*: $T \models \phi$ asks only whether ϕ is true in all models of T, with no mention of how we actually *derive* or *prove* ϕ from T. One of the deepest results in logic, Gödel's Completeness Theorem, shows that these two notions coincide: $T \models \phi$ if and only if there is a formal proof of ϕ from the axioms of T, written $T \vdash \phi$. In other words, truth across all models and provability from axioms turn out to be the same thing.

1.5 Proofs and Gödel's Completeness Theorem

So far, we have developed a purely *semantic* framework: we defined what it means for a formula to be *true* in a structure, and what it means for a sentence to be a logical consequence of a theory. But we have said nothing about *proofs* — the syntactic process of deriving conclusions from axioms by applying rules of inference.

In mathematics, we are used to proving things by writing down a sequence of steps, each following from the previous ones by some logical rule. We make this precise here.

Definition 1.29 (Formal Proof [1, Definition 3.1, pg. 44])**.** Let T be an $\mathcal{L}$-theory. A *formal proof* of a sentence ϕ from T is a finite *sequence* of assertions of the form:

1. $\gamma_1 \vdash \phi_1$
2. $\gamma_2 \vdash \phi_2$

$\vdots$ $\vdots$

n. $\gamma_n \vdash \phi_n$

where each assertion $\gamma_i \vdash \phi_i$ is justified by one of the following:

- (Axiom): $\phi_i \in T$, and $\gamma_i = \emptyset$. That is, any sentence in T may be asserted freely.
- (Assumption): $\phi_i \in \gamma_i$. That is, any formula in the current set of assumptions may be asserted.
- (Derived): $\gamma_i \vdash \phi_i$ follows from some earlier assertions $\gamma_j \vdash \phi_j$ and $\gamma_k \vdash \phi_k$ (with $j, k < i$) by one of the inference rules of first-order logic.

We write $T \vdash \phi$ if there exists a proof whose final assertion is $\emptyset \vdash \phi$, and say ϕ is *provable* from T.

Remark 1.30. The relation $T \vdash \phi$ is purely *syntactic*: it depends only on the symbols on the page, with no reference to truth in any structure. This stands in contrast to the semantic relation $T \models \phi$, which asks whether ϕ is true in every model of T.

At first glance, these two notions (provability and truth) seem to be quite different. Provability is about symbol manipulation; truth is about meaning in a structure. However, in 1930, Gödel showed in his Completeness Theorem that these two meanings coincide perfectly.

Theorem 1.31 (Gödel's Completeness Theorem, [2])**.** *Let T be an $\mathcal{L}$-theory and ϕ an $\mathcal{L}$-sentence. Then*

$$T \models \phi \iff T \vdash \phi.$$

That is, ϕ is true in every model of T if and only if ϕ has a formal proof from T.

Remark 1.32. The two directions of this theorem have two different interpretations.

1. **Soundness** ($\Leftarrow$): If ϕ is provable from T, then ϕ is true in every model of T. This essentially means that in every step of our assertion, truth is preserved. Essentially, this direction demonstrates soundness in our definition of proof.
2. **Completeness** ($\Rightarrow$): If ϕ is true in every model of T, then there is a formal proof of ϕ from T. This direction is a deep result. It says that our proof system is powerful enough to *capture all semantic truth*, that is, no true statement is left unprovable.

1.6 Homomorphisms, Embeddings and Isomorphisms

Having defined structures, we now study the maps between them. Just as group theory is enriched by the study of group homomorphisms, model theory is enriched by the study of *structure-preserving maps* between $\mathcal{L}$-structures. In this last subsection, we introduce the notion of homomorphisms and isomorphisms, which we use in our definition of interpretation in Chapter 4.2.

Definition 1.33 (Homomorphism)**.** Let $\mathcal{M}$ and $\mathcal{N}$ be $\mathcal{L}$-structures. A function $h : M \to N$ is an *$\mathcal{L}$-homomorphism* if:

1. For each constant symbol $c \in \mathcal{C}$: $h(c^{\mathcal{M}}) = c^{\mathcal{N}}$.
2. For each n-ary function symbol $f \in \mathcal{F}$ and all $a_1, \ldots, a_n \in M$:
$$h(f^{\mathcal{M}}(a_1, \ldots, a_n)) = f^{\mathcal{N}}(h(a_1), \ldots, h(a_n)).$$
3. For each n-ary relation symbol $R \in \mathcal{R}$ and all $a_1, \ldots, a_n \in M$:
$$(a_1, \ldots, a_n) \in R^{\mathcal{M}} \implies (h(a_1), \ldots, h(a_n)) \in R^{\mathcal{N}}.$$

Definition 1.34 ($\mathcal{L}$-Embedding)**.** An *$\mathcal{L}$-embedding* is a homomorphism $h : \mathcal{M} \to \mathcal{N}$ that is injective and satisfies the *strong* condition on relations:

$$(a_1, \ldots, a_n) \in R^{\mathcal{M}} \iff (h(a_1), \ldots, h(a_n)) \in R^{\mathcal{N}}.$$

Remark 1.35. Now, an $\mathcal{L}$-embedding is a stronger homomorphism which guarantees that:

- The homomorphism does not map two elements to the same element in $\mathcal{N}$, thereby "compressing" $\mathcal{M}$. — follows by *injectivity*
- The homomorphism does not map an element to something in $\mathcal{N}$ that satisfies a relation which the original element did not satisfy in $\mathcal{M}$, thereby "inventing" structure that was not there. — follows by the $\Leftarrow$ direction of the *strong condition* on relations.

Definition 1.36 (Isomorphism)**.** An *$\mathcal{L}$-isomorphism* is a surjective embedding. We write $\mathcal{M} \cong \mathcal{N}$ if there exists an isomorphism $\mathcal{M} \to \mathcal{N}$.

Lemma 1.37. *Let $h : \mathcal{M} \to \mathcal{N}$ be an $\mathcal{L}$-homomorphism. Then for any term $t(v_1, \ldots, v_n)$ and any $\bar{a} \in M^n$:*
$$h\big(t^{\mathcal{M}}[\bar{a}]\big) = t^{\mathcal{N}}[h(\bar{a})].$$

Proof. We proceed by induction on the complexity of terms t.
Suppose $t = c$ is a constant symbol. Then $t^{\mathcal{M}}[\bar{a}] = c^{\mathcal{M}}$, and by the homomorphism condition on constants, $h(c^{\mathcal{M}}) = c^{\mathcal{N}} = t^{\mathcal{N}}[h(\bar{a})]$. If $t = v_i$ is a variable, then $t^{\mathcal{M}}[\bar{a}] = a_i$, and $t^{\mathcal{N}}[h(\bar{a})] = h(a_i)$, so both sides equal $h(a_i)$.

Now, consider the term $t = f(t_1, \ldots, t_k)$. Assume the claim holds for each t_i. Then:

$$\begin{aligned}
h\big(t^{\mathcal{M}}[\bar{a}]\big) &= h\big(f^{\mathcal{M}}(t_1^{\mathcal{M}}[\bar{a}], \ldots, t_k^{\mathcal{M}}[\bar{a}])\big) \\
&= f^{\mathcal{N}}\big(h(t_1^{\mathcal{M}}[\bar{a}]), \ldots, h(t_k^{\mathcal{M}}[\bar{a}])\big) && \text{(homomorphism condition on } f\text{)} \\
&= f^{\mathcal{N}}\big(t_1^{\mathcal{N}}[h(\bar{a})], \ldots, t_k^{\mathcal{N}}[h(\bar{a})]\big) && \text{(induction hypothesis)} \\
&= t^{\mathcal{N}}[h(\bar{a})].
\end{aligned}$$

This completes the induction. □

Proposition 1.38 (Isomorphisms preserve truth)**.** *Let $h : \mathcal{M} \xrightarrow{\sim} \mathcal{N}$ be an isomorphism, and let $\phi(v_1, \ldots, v_n)$ be any $\mathcal{L}$-formula. Then for all $a_1, \ldots, a_n \in M$:*

$$\mathcal{M} \models \phi[a_1, \ldots, a_n] \iff \mathcal{N} \models \phi[h(a_1), \ldots, h(a_n)].$$

In particular, for any sentence σ: $\mathcal{M} \models \sigma \iff \mathcal{N} \models \sigma$.

Proof. We proceed by induction on the complexity of ϕ.

We first establish the atomic cases, making use of Lemma 1.37

- ϕ is $t_1 = t_2$. Then:

$$\begin{aligned}
\mathcal{M} \models (t_1 = t_2)[\bar{a}] &\iff t_1^{\mathcal{M}}[\bar{a}] = t_2^{\mathcal{M}}[\bar{a}] \\
&\iff h(t_1^{\mathcal{M}}[\bar{a}]) = h(t_2^{\mathcal{M}}[\bar{a}]) && \text{(injectivity of } h\text{)} \\
&\iff t_1^{\mathcal{N}}[h(\bar{a})] = t_2^{\mathcal{N}}[h(\bar{a})] && \text{Lemma 1.37} \\
&\iff \mathcal{N} \models (t_1 = t_2)[h(\bar{a})].
\end{aligned}$$

- ϕ is $R(t_1, \ldots, t_k)$ for some relation symbol R. Then:

$$\begin{aligned}
\mathcal{M} \models R(t_1, \ldots, t_k)[\bar{a}] &\iff \big(t_1^{\mathcal{M}}[\bar{a}], \ldots, t_k^{\mathcal{M}}[\bar{a}]\big) \in R^{\mathcal{M}} \\
&\iff \big(h(t_1^{\mathcal{M}}[\bar{a}]), \ldots, h(t_k^{\mathcal{M}}[\bar{a}])\big) \in R^{\mathcal{N}} && \text{(By embedding condition of } h\text{)} \\
&\iff \big(t_1^{\mathcal{N}}[h(\bar{a})], \ldots, t_k^{\mathcal{N}}[h(\bar{a})]\big) \in R^{\mathcal{N}} && \text{Lemma 1.37} \\
&\iff \mathcal{N} \models R(t_1, \ldots, t_k)[h(\bar{a})].
\end{aligned}$$

Inductive step. Suppose the claim holds for all formulas of strictly lower complexity than ϕ. We consider each way ϕ could have been formed.

- $\phi = \neg\psi$. By the induction hypothesis applied to ψ:

$$\mathcal{M} \models (\neg\psi)[\bar{a}] \iff \mathcal{M} \not\models \psi[\bar{a}] \iff \mathcal{N} \not\models \psi[h(\bar{a})] \iff \mathcal{N} \models (\neg\psi)[h(\bar{a})].$$

- $\phi = \psi_1 \wedge \psi_2$. By the induction hypothesis applied to each conjunct:

$$\begin{aligned}
\mathcal{M} \models (\psi_1 \wedge \psi_2)[\bar{a}] &\iff \mathcal{M} \models \psi_1[\bar{a}] \text{ and } \mathcal{M} \models \psi_2[\bar{a}] \\
&\iff \mathcal{N} \models \psi_1[h(\bar{a})] \text{ and } \mathcal{N} \models \psi_2[h(\bar{a})] \\
&\iff \mathcal{N} \models (\psi_1 \wedge \psi_2)[h(\bar{a})].
\end{aligned}$$

  The case $\phi = \psi_1 \vee \psi_2$ is identical.

- $\phi = \exists v\, \psi(v, v_1, \ldots, v_n)$. Then:

$$\begin{aligned}
\mathcal{M} \models (\exists v\, \psi)[\bar{a}] &\iff \exists b \in M,\ \mathcal{M} \models \psi[b, \bar{a}] \\
&\iff \exists b \in M,\ \mathcal{N} \models \psi[h(b), h(\bar{a})] && \text{(induction hypothesis)} \\
&\iff \exists c \in N,\ \mathcal{N} \models \psi[c, h(\bar{a})] && \text{(by surjectivity of } h\text{)} \\
&\iff \mathcal{N} \models (\exists v\, \psi)[h(\bar{a})].
\end{aligned}$$

- $\phi = \forall v\, \psi(v, v_1, \ldots, v_n)$. Then:

$$\begin{aligned}
\mathcal{M} \models (\forall v\, \psi)[\bar{a}] &\iff \forall b \in M,\ \mathcal{M} \models \psi[b, \bar{a}] \\
&\iff \forall b \in M,\ \mathcal{N} \models \psi[h(b), h(\bar{a})] && \text{(induction hypothesis)} \\
&\iff \forall c \in N,\ \mathcal{N} \models \psi[c, h(\bar{a})] && \text{(by surjectivity of } h\text{)} \\
&\iff \mathcal{N} \models (\forall v\, \psi)[h(\bar{a})].
\end{aligned}$$

  Alternatively, we can use the fact that $\forall v\ \psi \equiv \neg\exists v\ \neg\psi$ and use the previous cases to prove the universal quantifier case.

This completes the induction. □

Corollary 1.39. *Isomorphic structures satisfy exactly the same sentences:*

$$\mathcal{M} \cong \mathcal{N} \implies \mathrm{Th}(\mathcal{M}) = \mathrm{Th}(\mathcal{N})$$

Proof. Since sentences are classified as formulas, the implication follows from the above proposition. □

Example 1.40. Let $\mathcal{M} = (\mathbb{Z}, 0, 1, +, \cdot)$ and $\mathcal{N} = (2\mathbb{Z},\ 0,\ 2,\ \oplus,\ \otimes)$, where $2\mathbb{Z} = \{\ldots, -2, 0, 2, 4, \ldots\}$ is the set of even integers, with operations defined by $a \oplus b = a + b$ and $a \otimes b = a \cdot b/2$, and constants $0^{\mathcal{N}} = 0$ and $1^{\mathcal{N}} = 2$.

Define $h : \mathbb{Z} \to 2\mathbb{Z}$ by $h(n) = 2n$. One checks:

- $h(0) = 0 = 0^{\mathcal{N}}$ and $h(1) = 2 = 1^{\mathcal{N}}$,
- $h(a + b) = 2(a + b) = 2a \oplus 2b = h(a) \oplus h(b)$,
- $h(a \cdot b) = 2ab = (2a) \otimes (2b) = h(a) \otimes h(b)$.

So by definition, h is an isomorphism $\mathcal{M} \xrightarrow{\sim} \mathcal{N}$.

Now consider the sentence $\sigma \equiv \exists v\,(v + v = 1)$ ("there exists an element whose double is the multiplicative identity"). In $\mathcal{M}$, this asks whether $\frac{1}{2} \in \mathbb{Z}$, which is false: $\mathcal{M} \not\models \sigma$.

By the proposition, $\mathcal{N} \not\models \sigma$ either. Indeed, σ in $\mathcal{N}$ asks for $c \in 2\mathbb{Z}$ with $c \oplus c = 2$, i.e., $2c = 2$, i.e., $c = 1 \notin 2\mathbb{Z}$.

2 Computability

In the previous chapters, we built a precise language for making mathematical claims and a framework for evaluating their truth in structures. A natural question now arises: given a theory T and a sentence ϕ, can we *mechanically determine* whether $T \models \phi$? This is the *decision problem* for T. To make this question precise, we need a formal model of computation, that is, a rigorous notion of an *algorithm*.

2.1 Turing Machines and the Church–Turing Thesis

In 1936, Alan Turing formalized the intuitive notion of computation using a theoretical device now known as a **Turing machine**. Although simple in description, Turing machines are remarkably expressive as they are believed to capture everything that can be computed by any physical process.

Definition 2.1 (Turing Machine [3], Definition 3.3, pg. 168)**.** A *Turing Machine* is a tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ where:

1. Q is a finite set of *states*;
2. Σ is the finite *input alphabet* (not containing the blank symbol $\sqcup$);
3. $\Gamma \supseteq \Sigma \cup \{\sqcup\}$ is the finite *tape alphabet*;
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the *transition function*;
5. $q_0 \in Q$ is the *start state*;
6. $q_{\text{acc}}, q_{\text{rej}} \in Q$ are the *accepting* and *rejecting* states, with $q_{\text{acc}} \neq q_{\text{rej}}$.

The machine operates on an infinite tape, initially containing the input string followed by blanks. At each step, the machine reads the current tape symbol, writes a new symbol, moves the head left or right, and transitions to a new state. Computation ends when the machine enters q_{acc} or q_{rej}.

Remark 2.2 (Church-Turing Thesis)**.** For our purposes, the precise mechanics of Turing machines matter less than the class of functions and sets they compute. This is because of the *Church–Turing Thesis*, which says that every function that can be computed by an *effective procedure* (i.e., by any algorithm, in any reasonable model of computation) is computable by a Turing machine. Another way to phrase this is that all the proposed model of computation are equivalent to each other. The Church-Turing Thesis is universally accepted based on overwhelming evidence:

> Every proposed model of computation (lambda calculus, register machines, recursive functions, modern computers) has been shown to be equivalent to Turing machines.

2.2 Computable Functions and Sets

Since it is reasonable to consider the Church-Turing Thesis to be true, for simplicity sake, we will introduce computation using Turing machines for the rest of the report. However, it is worth noting to the reader that we could define the notion of computability using other algorithms.

Definition 2.3 (Computable Function [3, Definition 5.17 pg. 234])**.** A function $f : \mathbb{N}^k \to \mathbb{N}$ is *computable* if there exists a Turing machine M such that, on input $(n_1, \ldots, n_k)$ encoded as a string, M eventually halts and outputs $f(n_1, \ldots, n_k)$. In particular, M is required to halt on *every* input and such a function is also called *total*.

Definition 2.4 (Partial Computable Function)**.** A *partial computable* function $f : \mathbb{N}^k \rightharpoonup \mathbb{N}$ is one computed by a Turing machine that may fail to halt on some inputs (those inputs are outside the domain of f). We use the partial function notation $\rightharpoonup$ rather than $\to$.

Example 2.5 (Examples of computable and partial computable functions)**.**

1. The addition function $f : \mathbb{N}^2 \to \mathbb{N}$ defined by $f(n_1, n_2) = n_1 + n_2$ is *computable*. This is easy to see, because it will always halt and return $f(n_1, n_2) = n_1 + n_2$.

2. The halting function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ defined by

$$f(n) = \begin{cases} 1 & \text{if Turing machine } n \text{ halts on input } n \\ \text{undefined} & \text{otherwise} \end{cases}$$

   is *partial computable*. A Turing machine computes this by simulating machine n on input n and outputting 1 if it halts. However, if machine n loops forever, our simulation also loops forever .

Definition 2.6 (Computably Enumerable Set [3, Definition 3.5 pg. 170])**.** A set $A \subseteq \mathbb{N}$ is *computably enumerable* (denoted c.e.) if there exists a Turing machine M such that:

1. If $n \in A$, then M *eventually* halts and accepts on input n.

2. If $n \notin A$, then M either halts and rejects, or runs forever.

Definition 2.7 (Computable Set [3, Definition 3.6 pg. 170])**.** A set $A \subseteq \mathbb{N}$ is *computable* if there exists a Turing machine M such that for every $n \in \mathbb{N}$:

1. If $n \in A$, then M halts and accepts;

2. If $n \notin A$, then M halts and rejects.

Equivalently, A is computable iff its characteristic function $\chi_A : \mathbb{N} \to \{0, 1\}$ (where $\chi_A(n) = 1$ iff $n \in A$) is a computable function.

Remark 2.8. Sipser [3] calls computably enumerable sets *Turing-recognisable* and computable sets *Turing-decidable*. Both definitions are equivalent, that is, a set (or language) is computably enumerable (either accepts, rejects or loops) if a Turing machine can *recognise* it and computable (either accepts or rejects, but never loop) if a Turing machine can *decide* it.

Proposition 2.9 (Equivalent defintion of computable enumerable sets)**.** *The following are equivalent.*

1. *A is c.e.*

2. *A is the range of a computable function.*

3. *A is the domain of a partial computable function.*

Proof.

(1) $\Rightarrow$ (2): Suppose A is c.e., witnessed by Turing machine M. If $A = \emptyset$, take f to be any computable function with empty range. Otherwise, fix some $a_0 \in A$.

The key idea is this: rather than simulating M on a single input indefinitely, we simulate M on *all* inputs simultaneously, each for a bounded number of steps. Concretely, every

natural number k encodes a pair $\langle n, t\rangle$ via a computable pairing function, and we define $f : \mathbb{N} \to \mathbb{N}$ by:

$$f(\langle n, t\rangle) = \begin{cases} n & \text{if } M \text{ accepts } n \text{ within } t \text{ steps,} \\ a_0 & \text{otherwise.} \end{cases}$$

Since the simulation runs for at most t steps, f always halts, so f is total and computable. We verify $\text{range}(f) = A$:

- ($\subseteq$) $f(k)$ outputs either $a_0 \in A$, or some n accepted by M, hence $n \in A$. So $\text{range}(f) \subseteq A$.
- ($\supseteq$) For any $n \in A$, the machine M accepts n in some finite number of steps t_0, so $f(\langle n, t_0\rangle) = n$, giving $n \in \text{range}(f)$.

(2) $\Rightarrow$ (3): Suppose $A = \text{range}(f)$ for some total computable f. Define $g : \mathbb{N} \rightharpoonup \mathbb{N}$ by: on input n, enumerate $f(0), f(1), f(2), \ldots$ until some $f(k) = n$, then halt and output 1. If no such k exists, g runs forever. Then g is partial computable, and $g(n)$ is defined if and only if $n \in \text{range}(f) = A$, so $\text{dom}(g) = A$.

(3) $\Rightarrow$ (1): Suppose $A = \text{dom}(g)$ for some partial computable g, computed by machine M. Define M' to simply simulate M on input n and accept if M halts. Then M' accepts n if and only if $g(n)$ is defined, i.e., $n \in A$, so A is c.e. □

Example 2.10 (The Halting Problem [3, Theorem 5.11, pg. 216])**.** Define $H = \{(e, n) :$ Turing machine e halts on input $n\}$. This set H is computably enumerable (simulate machine e on input n; accept if it halts) but *not computable*. The non-computability of H is Turing's famous *undecidability theorem*, proved by a diagonal argument: if H were computable, one could construct a machine that halts iff it doesn't, a contradiction.

Remark 2.11. The distinction between computable and c.e. is critical: a computable set has a machine that always gives a definitive yes/no answer, whereas a c.e. set only has a machine that confirms membership, that is, if $n \notin A$, the machine may loop forever, never giving a negative answer. Every computable set is c.e., but the converse fails: the Halting Problem is c.e. but not computable.

The proposition above makes this gap precise. (2) explains the name: a c.e. set is one whose elements can be *listed out* by a machine, even if membership cannot be decided. (3) connects c.e. sets to partial computation, and will be used implicitly when we argue that a theory with a c.e. axiomatisation has its theorems enumerable by a machine.

Definition 2.12 (Decidable Theory)**.** A theory T is *decidable* if there exists an algorithm that, given any $\mathcal{L}$-sentence ϕ as input, correctly determines in finite time whether $T \models \phi$ or $T \not\models \phi$.

A theory T is *undecidable* if no such algorithm exists.

Remark 2.13.

1. It is important to the reader to notice that decidability is a property not just of a single structure $\mathcal{M}$, but of the complete first-order theory $\text{Th}(\mathcal{M})$.
2. To make the notion of decidability fully rigorous, one must encode sentences as natural numbers so that an algorithm (i.e., a Turing machine) can receive them as input. This is achieved via *Gödel numbering*, which we develop in Section 2.3. We will see there that the choice of encoding does not affect the notion of decidability.

The following relationship between decidability and the existence of a computable axiomatization is fundamental.

Proposition 2.14. *[1, Proposition 7.22 pg. 110] If T is a* complete *theory with a* computably enumerable *set of axioms, then T is decidable.*

Proof. By Gödel's Completeness Theorem, $T \models \phi$ iff there is a formal proof of ϕ from T. Since T is complete, for any sentence ϕ, either $T \models \phi$ or $T \models \neg\phi$.

Given ϕ, enumerate all proofs from T (this is possible since T is c.e. and proofs are finite syntactic objects). Run two enumerations in parallel: one searching for a proof of ϕ, and one searching for a proof of $\neg\phi$. By completeness, one of them will eventually be found. Output "true" if a proof of ϕ is found first, "false" otherwise. □

This proposition is the key mechanism behind Tarski's decidability result for real closed fields: we will show in Chapter 3 that RCF is complete and has a c.e. (in fact, finite) axiomatization, hence is decidable by Proposition 2.14.

Definition 2.15 (Reduction [3, Definition 5.20 pg. 235])**.** A set $A \subseteq \mathbb{N}$ is **many-one reducible** to $B \subseteq \mathbb{N}$, written $A \leq_m B$, if there exists a total computable function $f : \mathbb{N} \to \mathbb{N}$ such that for all n:

$$n \in A \iff f(n) \in B.$$

Reductions are the main tool for proving undecidability: to show a theory T is undecidable, one typically reduces the Halting Problem (or another known undecidable set) to $\mathrm{Th}(T)$.

Proposition 2.16. *If $A \leq_m B$ and B is computable (resp. c.e.), then so is A.*

Proof. Let $f : \mathbb{N} \to \mathbb{N}$ be the total computable function with $A \leq_m B$, so by definiton, $n \in A \iff f(n) \in B$. Suppose B is computable (resp. c.e.). We want to show that A is also computable (resp. c.e.).

On input $n \in \mathbb{N}$, we put the input through the function f (which is computable by definition), to get $f(n) \in \mathbb{N}$. Then, since B is computable, there is some Turing Machine M that halts or rejects based on membership of $f(n)$ in B. This procedure is then an effective computable procedure by a Turing Machine that halts or rejects based on membership of n in A. Hence, A is enumerable.

It is easy to see that this is still true if B is instead computably enumerable.

□

Remark 2.17. The proposition above provides a useful way to use reducibility of sets to determine computability.

2.3 Gödel Numbering

In order to apply computability theory to logic, we need to treat syntactic objects (formulas, sentences, proofs) that we have defined in Chapter 1 as inputs to Turing machines, which operate on strings of symbols. This is achieved via a method called **Gödel numbering**: a systematic encoding that assigns to each $\mathcal{L}$-formula ϕ a unique natural number $\ulcorner\phi\urcorner \in \mathbb{N}$, called its *Gödel number*.

The encoding must satisfy two key properties:

1. *Injectivity*: distinct formulas receive distinct numbers, so no information is lost.
2. *Computability*: ensuring that building $\neg\phi$, $\phi \wedge \psi$, $\exists v_i\, \phi$ from simpler formulas, checking whether a formula is a sentence and substituting terms for variables all correspond to computable operations on Gödel numbers.

Example 2.18 (Prime Power Encoding)**.** We describe one standard way to assign Gödel numbers for the language $\mathcal{L}_{\text{ring}} = \{0, 1, +, \cdot\}$.

We first assign codes to symbols. Fix the following bijection between the basic symbols and natural numbers:

Symbol	Code
v_i (variable)	$\langle 0, i\rangle$
0	1
1	2
$+$	3
$\cdot$	4
$=$	5
$\neg$	6
$\wedge$	7
$\exists$	8
(	9
)	10

where $\langle 0, i\rangle$ denotes some computable pairing of 0 and i into a single natural number (for example, $\langle a, b\rangle = 2^a \cdot 3^b$).

A formula is a finite sequence of symbols $(s_1, s_2, \ldots, s_k)$. So we encode this sequence as:

$$\ulcorner(s_1, \ldots, s_k)\urcorner = 2^{c_1} \cdot 3^{c_2} \cdot 5^{c_3} \cdots p_k^{c_k}$$

where c_i is the code of symbol s_i and p_k is the k-th prime. This is the standard *prime power encoding*.

For example, consider the atomic formula $\phi \equiv (v_1 = 0)$ in $\mathcal{L}_{\text{ring}}$. Its symbol sequence is ((, v_1 , $=$, 0 ,)), with codes $(9,\ \langle 0, 1\rangle,\ 5,\ 1,\ 10)$. The Gödel number is then:

$$\ulcorner\phi\urcorner = 2^9 \cdot 3^{\langle 0,1\rangle} \cdot 5^5 \cdot 7^1 \cdot 11^{10}.$$

This is a large but perfectly well-defined natural number, uniquely determined by ϕ. By the Fundamental Theorem of Arithmetic, the prime factorization uniquely recovers the original sequence of codes, so the encoding is injective.

Given $\ulcorner\phi\urcorner$, we also want to check whether ϕ has any free variables. Recall from Section 1 that only sentences have a definite truth value in a structure. We factor $\ulcorner\phi\urcorner$ to recover the symbol sequence, then scan through it tracking which variables are currently bound by a quantifier. If every variable occurrence falls within the scope of a matching quantifier, ϕ is a sentence; otherwise it is not.

Lastly, we need to verify computability of syntactic operations. $(\neg, \wedge, \vee, \exists, \forall)$ The key operations we need are all computable on Gödel numbers:

- $\neg$: Given $\ulcorner\phi\urcorner$, we want to compute $\ulcorner\neg\phi\urcorner$. Since $\neg\phi$ is just the symbol sequence of ϕ with $\neg$ prepended, we factor $\ulcorner\phi\urcorner$ to recover the sequence $(c_1, c_2, \ldots, c_k)$, shift each code up by one prime slot, and place $\texttt{code}(\neg)$ in the first slot. This is a straightforward arithmetic operation.

- $\wedge, \vee$: Given $\ulcorner\phi\urcorner$ and $\ulcorner\psi\urcorner$, we want to compute $\ulcorner\phi \wedge \psi\urcorner$. We factor both numbers to recover their symbol sequences, concatenate them with $\texttt{code}(\wedge)$ inserted in between, and re-encode the resulting sequence as a prime power product. The $\vee$ case is identical.

- $\forall, \exists$: Given $\ulcorner\phi\urcorner$ and a variable v_i, we want to compute $\ulcorner\exists v_i\, \phi\urcorner$. We prepend $\texttt{code}(\exists)$ and $\texttt{code}(v_i)$ to the symbol sequence of ϕ. The case $\forall$ is identical.

All of these operations reduce to factoring and arithmetic on natural numbers, which are computable by Turing machines. This confirms that Gödel numbering successfully bridges syntax and computation: from this point on, a Turing machine can receive $\ulcorner\phi\urcorner$ as input and manipulate ϕ entirely through arithmetic.

Remark 2.19. It is worth noting that this encoding above is just one of many possible ways to assign Gödel numbers. We could use another method (like *binary encoding* to assign Gödel numbers.) This raises a natural question: Does the choice of assignment of Gödel numbers affect the decidability of theories?

The answer is no, and we show this in the proposition below.

Proposition 2.20 (Equivalence of Gödel Numberings)**.** *Let g_1 and g_2 be two Gödel numberings of $\mathcal{L}$-formulas, both satisfying injectivity and computability. Then for any theory T, the set $\{g_1(\phi) : T \models \phi\}$ is computable if and only if $\{g_2(\phi) : T \models \phi\}$ is computable.*

Proof. Define the *translation function* $f : \mathbb{N} \to \mathbb{N}$ by $f(g_1(\phi)) = g_2(\phi)$ for each formula ϕ. Since both g_1 and g_2 are computable and injective, f is a well-defined total computable function: given $g_1(\phi)$, we can decode ϕ from g_1 (by running the g_1-decoder, which is computable) and then re-encode it under g_2 (by running the g_2-encoder, which is computable).

Now let $A_1 = \{g_1(\phi) : T \models \phi\}$ and $A_2 = \{g_2(\phi) : T \models \phi\}$. Then:

$$n \in A_1 \iff f(n) \in A_2$$

so $A_1 \leq_m A_2$ via f. Symmetrically, the inverse translation gives $A_2 \leq_m A_1$. Hence A_1 is computable if and only if A_2 is computable (since many-one reductions preserve and reflect computability in both directions). □

Remark 2.21. This proposition allows us to not need to specify which encoding we use, because all reasonable encodings yield the same notion of decidability. What matters is only that the encoding is injective and computable, not how the specific numbers assigned.

This is what makes the statement of decidability precise: a theory T is decidable if and only if the set $\{\ulcorner\phi\urcorner : T \models \phi\}$ is a computable set of natural numbers. Without Gödel numbering, we could not even pose the decision problem as a question about computation.

3 Quantifier Elimination

The central tool connecting the semantic world of model theory to the computational world of decidability is **quantifier elimination**. Intuitively, we say that a theory admits quantifier elimination if every formula, no matter how many alternating quantifiers it contains, is equivalent (within the theory) to one with no quantifiers at all.

The usefulness of quantifier elimination in evaluating the truth values of formulas is demonstrated in the following example.

Example 3.1. Consider the formula

$$\phi \;\equiv\; \exists v_1 \, \exists v_2 \, (v_1 \cdot v_1 + v_2 \cdot v_2 = 0)$$

in the theory of ordered fields, asking whether there exist two elements whose squares sum to zero. Naively, checking this requires searching over all pairs (v_1, v_2) in the field, which possibly involves an infinite number of steps. This causes a problem for us because as defined in Chapter 2, our algorithms must accept or halt in *finite* number of sets.

Thankfully, we are able to use the following fact: In any ordered field, $v_1 \cdot v_1 \geq 0$ and $v_2 \cdot v_2 \geq 0$, so their sum is zero if and only if both are zero. The formula is therefore equivalent to the quantifier-free sentence

$$\phi^* \;\equiv\; 0 = 0 \wedge 0 = 0,$$

which is just the statement that $v_1 = 0$ and $v_2 = 0$ is the only solution. A computer can verify ϕ^* instantly, with no search over the domain required. This is the key computational payoff of quantifier elimination: an infinite search is immediately shortened to a 2-step process.

3.1 Quantifier Elimination: Definition and Consequences

Definition 3.2 (Quantifier-free Formula)**.**

1. An $\mathcal{L}$-formula ϕ is *quantifier-free* if it contains no occurrences of $\exists$ or $\forall$.

2. Recall our construction of $\mathcal{L}$-formulas. This gives us an alternate definition:

   ϕ is quantifier-free if is built from atomic formulas using only the Boolean connectives $\neg, \wedge, \vee$ (that is, not built by quantifiers $\exists$ and $\forall$), by our definition of an $\mathcal{L}$-formula.

Definition 3.3 (Quantifier Elimination)**.** A theory T admits *quantifier elimination* (QE) if for every $\mathcal{L}$-formula $\phi(\bar{v})$, there exists a quantifier-free $\mathcal{L}$-formula $\phi^*(\bar{v})$ with the same free variables such that

$$T \models \forall \bar{v} \, \big(\phi(\bar{v}) \leftrightarrow \phi^*(\bar{v})\big).$$

That is, ϕ and ϕ^* are logically equivalent in every model of T.

Remark 3.4. It suffices to show QE for existential formulas of the form $\exists v \, \psi(\bar{u}, v)$, where ψ is quantifier-free. Indeed, universal formulas $\forall v \, \psi$ are equivalent to $\neg\exists v \, \neg\psi$, and arbitrary formulas are built from such existential blocks. So in practice, proving QE reduces to showing that a single existential quantifier can always be eliminated.

Theorem 3.5 (QE implies Completeness and Decidability)**.** *Let T be an $\mathcal{L}$-theory that admits quantifier elimination. Suppose further that for every quantifier-free sentence σ (i.e., a quantifier-free formula with no free variables), either $T \models \sigma$ or $T \models \neg\sigma$. Then:*

1. *T is **complete**.*

2. *If T has a c.e. axiomatization, then T is **decidable**.*

Proof. (1) Let σ be any sentence. By QE, $T \models \sigma \leftrightarrow \sigma^*$ where σ^* is quantifier-free. Since σ^* has no free variables, it is a quantifier-free sentence, and by hypothesis either $T \models \sigma^*$ or $T \models \neg\sigma^*$. Thus either $T \models \sigma$ or $T \models \neg\sigma$, so T is complete.

(2) By (1), T is complete and has a c.e. axiomatization; Proposition 2.14 gives decidability. □

3.2 General Strategy for Quantifier Elimination

The key insight that makes quantifier elimination tractable in practice is that arbitrary formulas can be reduced to a normal form, after which it suffices to handle a single existential quantifier applied to a conjunction of literals. We develop this reduction here.

Definition 3.6 (Literal)**.** A *literal* is either an atomic $\mathcal{L}$-formula or the negation of one.

Definition 3.7 (Disjunctive Normal Form [1, Definition 1.23, pg.14])**.** A quantifier-free formula is in *disjunctive normal form* (DNF) if it is of the form

$$\bigvee_{i=1}^{n} \bigwedge_{j=1}^{m_i} \phi_{ij}$$

where each ϕ_{ij} is a literal, that is, either atomic or the negation of an atomic formula.

Proposition 3.8 ([1, Corollary 1.25, pg.14])**.** *Every quantifier-free formula ϕ is equivalent to a formula ϕ^* in disjunctive normal form.*

We can now describe the general strategy. By the remark after Definition 3.3, it suffices to eliminate a single existential quantifier from a formula of the form $\exists x\, \phi(x, \bar{y})$, where ϕ is quantifier-free. The reduction then proceeds as follows.

Proposition 3.9 (Reduction to a conjunction of literals)**.** *To show that a theory T admits quantifier elimination, it suffices to show that for every conjunction of literals $\bigwedge_{i\in I} \phi_i(x, \bar{y})$, the formula $\exists x \bigwedge_{i\in I} \phi_i(x, \bar{y})$ is equivalent to a quantifier-free formula.*

Proof. Let $\phi(x, \bar{y})$ be any quantifier-free formula. By Proposition 3.8, we may write ϕ in disjunctive normal form:

$$\phi(x, \bar{y}) \;\equiv\; \bigvee_{j\in J} \bigwedge_{i\in I_j} \phi_{ij}(x, \bar{y}),$$

where each ϕ_{ij} is a literal. Since $\exists$ distributes over $\vee$, we have

$$\exists x\, \phi(x, \bar{y}) \;\equiv\; \bigvee_{j\in J} \exists x \bigwedge_{i\in I_j} \phi_{ij}(x, \bar{y}).$$

If each conjunct $\exists x \bigwedge_{i\in I_j} \phi_{ij}(x, \bar{y})$ can be replaced by a quantifier-free formula $\phi_j^*(\bar{y})$, then

$$\exists x\, \phi(x, \bar{y}) \;\equiv\; \bigvee_{j\in J} \phi_j^*(\bar{y}),$$

which is quantifier-free. □

Remark 3.10. We will use this strategy in our approach of showing quantifier elimination for the different theories in this report.

3.3 Quantifier Elimination for Presburger Arithmetic

Before tackling the more complex case of real closed fields, we warm up with a simpler but illustrative example: the theory of $(\mathbb{Z}, +)$.

Definition 3.11 (Presburger Arithmetic)**.** We work in the language $\mathcal{L}_{\text{Pres}} = \{+, 0, 1, \equiv_2, \equiv_3, \ldots\}$, where each $\equiv_n$ is a binary relation symbol interpreted as congruence modulo n. The theory of *Presburger arithmetic* is the $\mathcal{L}_{\text{Pres}}$-theory axiomatized by:

(A) the axioms of abelian groups,

(B) $\forall x,\ x + 0 = x$ and $1 + 1 + \cdots + 1 \neq 0$ for each finite sum (characteristic zero),

(C) $\forall x \forall y,\ x \equiv_n y \leftrightarrow \exists z,\ x = y + n \cdot z$ for each $n \geq 2$ (definition of congruences).

Remark 3.12. The reason we need to expand the language with congruence relations $\equiv_n$ is that quantifier elimination *fails* in the smaller language $\{+, 0, 1\}$ alone. For instance, the formula $\exists y,\ x = y + y$ ("x is even") has no quantifier-free equivalent in $\{+, 0, 1\}$, but becomes $x \equiv_2 0$ once we add the congruence relations.

Theorem 3.13 (Presburger, 1929)**.** T_{Pres} *admits quantifier elimination in* $\mathcal{L}_{\text{Pres}}$*.*

Proof. As before, it suffices to eliminate $\exists x$ from a conjunction of literals. In $\mathcal{L}_{\text{Pres}}$, the literals involving x are of the following forms:

1. *Equations*: $x + p_i(\bar{y}) = 0$,
2. *Inequations*: $x + q_i(\bar{y}) \neq 0$,
3. *Congruences*: $x \equiv_n r_i(\bar{y})$ or $x \not\equiv_n s_i(\bar{y})$,

where $p_i(\bar{y}), q_i(\bar{y}), r_i(\bar{y}), s_i(\bar{y})$ are linear combinations of parameters. We deal with each case.

Equations. Suppose $x + p_k(\bar{y}) = 0$ for some k. Then, we can move everything to one side of the equality to get $x = -p_k(\bar{y})$. Substituting $x = p_k(\bar{y})$ into all other literals eliminates x and we end up with quantifier-free formulas for the equations i.e. $x + p_i(\bar{y}) \equiv -p_k(\bar{y}) + p_i(\bar{y}) = 0$ which is quantifier-free.

Now, since we have dealt with the case of equations, x is now only constrained by congruences and inequations.

Congruences. The positive congruences $x \equiv_{n_1} r_1(\bar{y}),\ \ldots,\ x \equiv_{n_k} r_k(\bar{y})$ form a system of simultaneous linear congruences. By the Chinese Remainder Theorem, this system has a solution if and only if for each pair i, j,

$$r_i(\bar{y}) \equiv_{\gcd(n_i, n_j)} r_j(\bar{y}),$$

which is a quantifier-free condition on $\bar{y}$. When it is satisfied, the solutions form a single residue class modulo $N := \operatorname{lcm}(n_1, \ldots, n_k)$, that is, all integers of the form $x_0(\bar{y}) + Nm$ for $m \in \mathbb{Z}$, where $x_0(\bar{y})$ is a particular solution. The negative congruences $x \not\equiv_{n_i} s_i(\bar{y})$ each rule out finitely many residue classes modulo n_i, and whether any of these conflict with the solution class modulo N is also clearly a quantifier-free condition.

Inequations. Each inequation $x + q_i(\bar{y}) \neq 0$ forbids x from taking the single value $-q_i(\bar{y})$. Since the solution set of the congruence system is an infinite arithmetic progression (whenever it is nonempty), and we are only forbidding finitely many values, there always remains a solution. Therefore the inequations impose no further constraint on *existence*, and can be dropped entirely. □

Example 3.14. To better understand the QE methodology of the theory of Presburger arithmetic, we will give an example where we eliminate a quantifier-bound formula.

Consider the formula

$$\varphi(x, y_1, y_2, y_3) \;=\; \exists x \left[x + y_1 \neq 0 \;\wedge\; x \equiv_2 y_2 \;\wedge\; x \equiv_4 y_3 \;\wedge\; x \not\equiv_4 1 \right].$$

Equations. There are no equations.

Congruences. The positive congruences are $x \equiv_2 y_2$ and $x \equiv_4 y_3$. By CRT, a solution exists if and only if

$$y_2 \;\equiv_{\gcd(2,4)}\; y_3, \qquad \text{i.e.} \qquad y_2 \equiv_2 y_3,$$

which is a genuine quantifier-free condition. When satisfied, the solutions form a residue class modulo $N = \operatorname{lcm}(2, 4) = 4$, namely $x \equiv_4 y_3$.

Now the negative congruence $x \not\equiv_4 1$ rules out the residue class $1 \pmod 4$ from the solution class $y_3 \pmod 4$. This conflicts with the solution class if and only if $y_3 \equiv_4 1$, which is again a quantifier-free condition. So the combined congruence system has a solution if and only if

$$(y_2 \equiv_2 y_3) \;\wedge\; (y_3 \not\equiv_4 1).$$

Step 3 (Inequations). The inequation $x + y_1 \neq 0$ forbids the single value $x = -y_1$. The solution set is the infinite arithmetic progression $\{y_3 + 4m : m \in \mathbb{Z}\}$. Removing one point still leaves infinitely many solutions, so the inequation imposes no constraint on existence and is dropped.

Result. The formula $\varphi(y_1, y_2, y_3)$ is equivalent to

$$\varphi'(y_1, y_2, y_3) \;=\; (y_2 \equiv_2 y_3) \;\wedge\; (y_3 \not\equiv_4 1).$$

Note that y_1 drops out entirely, confirming that the inequation places no constraint on the parameters.

Corollary 3.15. $\mathrm{Pres} = \mathrm{Th}(\mathbb{Z}, +, 0, 1, \equiv_2, \equiv_3, \ldots)$. *In particular, the first-order theory of* $(\mathbb{Z}, +)$ *is decidable.*

3.4 Quantifier Elimination for Real Closed Fields

We now turn to the main theorem of this chapter: the theory of real closed fields admits quantifier elimination. This result is credited to Tarski ([4], 1951).

Definition 3.16 (Ordered Field)**.** An *ordered field* is an $\mathcal{L}_{\mathrm{or}}$-structure $(F, 0, 1, +, \cdot, <)$ satisfying the field axioms together with:

1. $<$ is a linear order compatible with the field operations: $a < b \implies a + c < b + c$, and $0 < a, 0 < b \implies 0 < a \cdot b$.

In any ordered field, we have $0 < 1$, all squares are non-negative ($0 \leq a^2$), and -1 is not a sum of squares.

Definition 3.17 (Real Closed Field)**.** An ordered field $(F, 0, 1, +, \cdot, <)$ is a *real closed field* if:

1. Every positive element has a square root: $\forall a \, (0 < a \to \exists b \, (b \cdot b = a))$.
2. Every polynomial of odd degree has a root: for each $n \geq 1$,

$$\forall a_0, \ldots, a_{2n} \, \exists x \left(a_{2n} \cdot x^{2n+1} + \cdots + a_1 \cdot x + a_0 = 0 \right).$$

The *theory of real closed fields*, denoted RCF, is the $\mathcal{L}_{\mathrm{or}}$-theory axiomatized by the ordered field axioms together with (1) and all instances of (2).

Theorem 3.18 (QE for RCF [4])**.** *The theory* RCF *admits quantifier elimination (in the language* $\mathcal{L}_{\text{or}} = \{0, 1, +, \cdot, <\}$*). That is, for every* $\mathcal{L}_{\text{or}}$*-formula* $\phi(\bar{v})$*, there exists a quantifier-free* $\mathcal{L}_{\text{or}}$*-formula* $\phi^*(\bar{v})$ *such that* RCF $\models \phi \leftrightarrow \phi^*$.

Proof sketch. By the remark after Definition 3.3, it suffices to eliminate a single existential quantifier. That is, we need to show: given a quantifier-free formula $\psi(v, \bar{u})$, there exists a quantifier-free $\chi(\bar{u})$ such that RCF $\models \exists v\, \psi(v, \bar{u}) \leftrightarrow \chi(\bar{u})$.

First, we notice that every quantifier-free formula in $\mathcal{L}_{\text{or}}$ is a Boolean combination of atomic formulas of the forms $p(\bar{u}, v) = 0$ and $p(\bar{u}, v) > 0$, where p is a polynomial with integer coefficients. Writing ψ in disjunctive normal form, it suffices to handle a single conjunct of the form:

$$p_1(v) = 0 \wedge \cdots \wedge p_r(v) = 0 \wedge q_1(v) > 0 \wedge \cdots \wedge q_s(v) > 0$$

The key algebraic fact is the following: whether a system of polynomial equalities and strict inequalities has a real solution can be determined by a quantifier-free condition on the coefficients. This is accomplished via the theory of *Sturm sequences* and *Sylvester–Habicht sequences*, which give effective methods for counting the number of real roots of a polynomial in a given interval. (Understanding and proving this is not required).

Therefore, given polynomials $p_1, \ldots, p_r, q_1, \ldots, q_s \in F[v]$ (where F is a real closed field and the coefficients are elements of F), one can write a quantifier-free condition χ on the coefficients such that:

$$F \models \exists v \left(\bigwedge_i p_i(v) = 0 \wedge \bigwedge_j q_j(v) > 0 \right) \iff F \models \chi.$$

For an arbitrary formula with multiple nested quantifiers $\exists v_1\, \exists v_2 \cdots \psi$, we apply the single-quantifier elimination repeatedly from the inside out, each time replacing the innermost existential with a quantifier-free equivalent. The resulting procedure terminates and produces a quantifier-free formula equivalent to the original, confirming that RCF admits quantifier elimination. □

Theorem 3.19 (Completeness and Decidability of RCF)**.** RCF *is complete and decidable.*

Proof. By Theorem 3.18, RCF admits QE. The only quantifier-free sentences in $\mathcal{L}_{\text{or}}$ are Boolean combinations of polynomial equalities and inequalities between integer constants (since there are no free variables), e.g., $2 > 1$, $0 = 0$, $1 + 1 = 2$. All such sentences are clearly determined by the ordered field axioms alone. Hence by Theorem 3.5, RCF is complete.

Since RCF has a finite (and in fact, computably enumerable) axiom scheme and is complete, decidability follows from Proposition 2.14. □

4 Decidability of Euclidean Geometry

We have now assembled all the ingredients needed for one of the most beautiful results in mathematical logic, that Euclidean Geometry is in fact *decidable*. The strategy is elegant, because we do not prove decidability of geometry directly. Instead, we show that Euclidean geometry can be *interpreted* inside the theory of real closed fields, and then we use the fact that RCF is decidable (Theorem 3.19) to *transfer* decidability.

This approach (due to Tarski) follows in three steps:

1. Define formally what it means for one theory to be *interpretable* in another.
2. Prove that if $\mathcal{M}$ is decidable and $\mathcal{N}$ is interpretable in $\mathcal{M}$, then $\mathcal{N}$ is decidable.
3. Show that Euclidean geometry is interpretable in RCF, and hence Euclidean geometry is decidable.

4.1 The Language of Euclidean Geometry

We first fix a formal language for Euclidean plane geometry. Tarski's approach encodes geometry in a surprisingly minimal language.

Definition 4.1 (Language of Geometry)**.** We denote the language of elementary Euclidean geometry as $\mathcal{L}_{\text{geo}} = \{\mathrm{B}, \equiv\}$, where:

1. $\mathrm{B}(x, y, z)$ is a ternary relation symbol meaning "y lies *between* x and z" (on a line segment, inclusive of endpoints).
2. $x\,y \equiv z\,w$ is a quaternary relation symbol meaning "the distance from x to y equals the distance from z to w" (equidistance).

The variables of $\mathcal{L}_{\text{geo}}$ range over *points* in the plane.

Tarski showed that the *true* sentences of Euclidean plane geometry, that is, sentences in $\mathcal{L}_{\text{geo}}$ that hold in the standard Euclidean plane $\mathbb{R}^2$, form a complete, axiomatizable theory. We denote this theory EG (short for Elementary Geometry). The key to proving EG is decidable is to interpret it inside RCF.

4.2 Interpretations

The notion of an interpretation formalises what it means to "translate" one theory into another. We use Hodges' definition on interpretability.

Definition 4.2 (Interpretation [5])**.** Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be languages, T_1 an $\mathcal{L}_1$-theory and T_2 an $\mathcal{L}_2$-theory. An *n-dimensional interpretation* $\mathcal{I}$ of T_1 in T_2 consists of:

1. **Domain formula.** An $\mathcal{L}_2$-formula $\delta(x_1, \ldots, x_n)$ such that $T_2 \vdash \exists x_1 \cdots x_n\, \delta(x_1, \ldots, x_n)$. (to ensure that the interpreted structure we get is non-empty.) The formula δ selects the n-tuples of the T_2-universe that serve as *coordinates* for elements of the interpreted structure.
2. **Equality formula.** An $\mathcal{L}_2$-formula $\varepsilon(\bar{x}, \bar{y})$ (where $\bar{x} = x_1, \ldots, x_n$ and $\bar{y} = y_1, \ldots, y_n$ are disjoint n-tuples of variables) such that T_2 proves that ε defines an equivalence relation on the set $\delta(\mathcal{M}^n)$ of all n-tuples satisfying δ in every model $\mathcal{M}$ of T_2.

   The equivalence class of an n-tuple $\bar{a}$ under ε is written $\bar{a}\,/_{\varepsilon}$; these classes are the *elements* of the interpreted structure.

3. **Relation symbols.** For each k-ary relation symbol $R \in \mathcal{L}_1$, an $\mathcal{L}_2$-formula $\varphi_R(\bar{x}_0, \ldots, \bar{x}_{k-1})$ (where each $\bar{x}_i$ is itself a n-tuple of variables) such that T_2 proves that φ_R is *compatible* with ε:

$$\varepsilon(\bar{x}_0, \bar{y}_0) \wedge \cdots \wedge \varepsilon(\bar{x}_{k-1}, \bar{y}_{k-1}) \wedge \delta(\bar{x}_0) \wedge \cdots \wedge \delta(\bar{x}_{k-1}) \;\longrightarrow\; \big(\varphi_R(\bar{x}_0, \ldots, \bar{x}_{k-1}) \leftrightarrow \varphi_R(\bar{y}_0, \ldots, \bar{y}_{k-1})\big).$$

   Thus φ_R is well-defined on equivalence classes.

4. **Function symbols.** For each k-ary function symbol $f \in \mathcal{L}_1$, an $\mathcal{L}_2$-formula $\psi_f(\bar{x}_0, \ldots, \bar{x}_{k-1}, \bar{y})$ such that T_2 proves:

   - *Totality and closure*: $\forall \bar{x}_0 \cdots \bar{x}_{k-1} \big(\delta(\bar{x}_0) \wedge \cdots \wedge \delta(\bar{x}_{k-1}) \to \exists \bar{y}\, (\delta(\bar{y}) \wedge \psi_f(\bar{x}_0, \ldots, \bar{x}_{k-1}, \bar{y}))\big)$.
   - *Functionality up to* ε: $\forall \bar{x}_0 \cdots \bar{x}_{k-1}\, \forall \bar{y}\, \forall \bar{z} \big(\psi_f(\bar{x}_0, \ldots, \bar{x}_{k-1}, \bar{y}) \wedge \psi_f(\bar{x}_0, \ldots, \bar{x}_{k-1}, \bar{z}) \to \varepsilon(\bar{y}, \bar{z})\big)$.
   - *Compatibility with* ε: if ε identifies the inputs then ε identifies the outputs.

5. **Constant symbols.** For each constant symbol $c \in \mathcal{L}_1$, an $\mathcal{L}_2$-formula $\psi_c(\bar{y})$ such that T_2 proves existence of a unique (up to ε) witness in the domain: $\exists \bar{y}\, (\delta(\bar{y}) \wedge \psi_c(\bar{y}))$ and $\forall \bar{y}\, \forall \bar{z}\, (\psi_c(\bar{y}) \wedge \psi_c(\bar{z}) \to \varepsilon(\bar{y}, \bar{z}))$.

6. **Translation map $\mathcal{I}$.** A recursive map from $\mathcal{L}_1$-formulas to $\mathcal{L}_2$-formulas defined as follows. Write $\bar{x}^{\mathcal{I}}$ for an n-tuple of fresh $\mathcal{L}_2$-variables corresponding to the $\mathcal{L}_1$-variable x.

   - *Equality*: $(t_1 = t_2)^{\mathcal{I}} \;=\; \varepsilon(t_1^{\mathcal{I}}, t_2^{\mathcal{I}})$.
   - *Relations*: $R(t_1, \ldots, t_k)^{\mathcal{I}} \;=\; \varphi_R(t_1^{\mathcal{I}}, \ldots, t_k^{\mathcal{I}})$.
   - *Boolean connectives*: $(\neg\phi)^{\mathcal{I}} = \neg\phi^{\mathcal{I}}, \quad (\phi \wedge \psi)^{\mathcal{I}} = \phi^{\mathcal{I}} \wedge \psi^{\mathcal{I}}$, etc.
   - *Quantifiers*:

$$\begin{aligned} (\exists x\, \phi)^{\mathcal{I}} &\;=\; \exists \bar{x}^{\mathcal{I}} \left(\delta(\bar{x}^{\mathcal{I}}) \wedge \phi^{\mathcal{I}}\right), \\ (\forall x\, \phi)^{\mathcal{I}} &\;=\; \forall \bar{x}^{\mathcal{I}} \left(\delta(\bar{x}^{\mathcal{I}}) \to \phi^{\mathcal{I}}\right). \end{aligned}$$

The formulas δ, ε, and all φ_R, ψ_f, ψ_c are called the *defining formulas* of $\mathcal{I}$; the sentences asserting admissibility, compatibility, etc. are called the *admissibility conditions* of $\mathcal{I}$.

Definition 4.3 (The interpreted structure $\mathcal{N}^{\mathcal{I}}$)**.** Let $\mathcal{I}$ be an n-dimensional interpretation as above, and let $\mathcal{M} \models T_2$ satisfy the admissibility conditions of $\mathcal{I}$. The *$\mathcal{I}$-structure induced by* $\mathcal{M}$, written $\mathcal{N}^{\mathcal{I}}$ is defined as follows:
Universe: $N^{\mathcal{I}} = \delta(\mathcal{M}^n)/_{\varepsilon^{\mathcal{M}}}$, the set of ε-equivalence classes of n-tuples satisfying δ in $\mathcal{M}$.
Relations: for a k-ary $R \in \mathcal{L}_1$,

$$R^{\mathcal{N}^{\mathcal{I}}}(\bar{a}_0/_{\varepsilon}, \ldots, \bar{a}_{k-1}/_{\varepsilon}) \iff \mathcal{M} \models \varphi_R(\bar{a}_0, \ldots, \bar{a}_{k-1}).$$

Functions and constants: defined analogously using ψ_f and ψ_c.

And these definitions are independent of the choice of representatives $\bar{a}_i$ from their equivalence classes.

4.3 The Truth Lemma and Transfer of Decidability

The following Truth Lemma is the key result connecting interpretations to truth.

Lemma 4.4 (Truth Lemma [5])**.** *Let $\mathcal{M} \models T_2$ be a structure satisfying the* admissibility conditions *of an n-dimensional interpretation $\mathcal{I}$ of a language $\mathcal{L}_1$ into a language $\mathcal{L}_2$. Let $\mathcal{N}^{\mathcal{I}}$ be the interpreted structure with universe $N^{\mathcal{I}} = \delta(\mathcal{M}^n)/_{\varepsilon}$.*

Let $f_{\mathcal{I}} \colon \delta(\mathcal{M}^n) \twoheadrightarrow N^{\mathcal{I}}$ be the surjective coordinate map *defined by:*

$$\bar{a} \mapsto \bar{a}/_{\varepsilon}$$

Then for every $\mathcal{L}_1$-formula $\theta(x_1, \ldots, x_k)$ and every k-tuple of n-tuples $\bar{a}_1, \ldots, \bar{a}_k \in \delta(\mathcal{M}^n)$, the following equivalence holds:

$$\mathcal{N}^{\mathcal{I}} \models \theta\,(\bar{a}_1/_{\varepsilon}, \ldots, \bar{a}_k/_{\varepsilon}) \iff \mathcal{M} \models \theta^{\mathcal{I}}(\bar{a}_1, \ldots, \bar{a}_k)$$

In particular, for every $\mathcal{L}_1$-sentence σ:

$$\mathcal{N}^{\mathcal{I}} \models \sigma \iff \mathcal{M} \models \sigma^{\mathcal{I}}$$

Proof. We proceed by structural induction on θ.

Atomic cases.

- θ is $t_1 = t_2$. By definition of $\mathcal{N}^{\mathcal{I}}$, equality holds iff the representing tuples are ε-equivalent, which is exactly $\mathcal{M} \models \varepsilon(\bar{a}_1, \bar{a}_2) = (t_1 = t_2)^{\mathcal{I}}(\bar{a}_1, \bar{a}_2)$.
- θ is $R(t_1, \ldots, t_k)$. By definition of $R^{\mathcal{N}^{\mathcal{I}}}$ and the compatibility condition,

$$\mathcal{N}^{\mathcal{I}} \models R\big(\tfrac{\bar{a}_1}{\varepsilon}, \ldots, \tfrac{\bar{a}_k}{\varepsilon}\big) \iff \mathcal{M} \models \varphi_R(\bar{a}_1, \ldots, \bar{a}_k) = \theta^{\mathcal{I}}(\bar{a}_1, \ldots, \bar{a}_k).$$

Inductive cases. Suppose the lemma holds for all formulas of strictly lower complexity.

- $\theta = \neg\psi$. Then

$$\begin{aligned} \mathcal{N}^{\mathcal{I}} \models (\neg\psi)[\bar{a}/\varepsilon] &\iff \mathcal{N}^{\mathcal{I}} \not\models \psi[\bar{a}/\varepsilon] \\ &\iff \mathcal{M} \not\models \psi^{\mathcal{I}}[\bar{a}] && \text{(induction hypothesis)} \\ &\iff \mathcal{M} \models (\neg\psi)^{\mathcal{I}}[\bar{a}]. \end{aligned}$$

- $\theta = \psi_1 \wedge \psi_2$. Follows immediately from the induction hypothesis applied to each conjunct.
- $\theta = \exists x\, \psi(x, x_1, \ldots, x_k)$. Then:

$$\begin{aligned} &\mathcal{N}^{\mathcal{I}} \models (\exists x\, \psi)\big[\tfrac{\bar{a}_1}{\varepsilon}, \ldots, \tfrac{\bar{a}_k}{\varepsilon}\big] \\ &\iff \exists \bar{c}/\varepsilon \in N^{\mathcal{I}},\ \mathcal{N}^{\mathcal{I}} \models \psi\big[\tfrac{\bar{c}}{\varepsilon}, \tfrac{\bar{a}_1}{\varepsilon}, \ldots, \tfrac{\bar{a}_k}{\varepsilon}\big] \\ &\iff \exists \bar{c} \in \delta(\mathcal{M}^n),\ \mathcal{M} \models \psi^{\mathcal{I}}[\bar{c}, \bar{a}_1, \ldots, \bar{a}_k] \quad \text{(induction hypothesis, using surjectivity of } f_{\mathcal{I}}) \\ &\iff \mathcal{M} \models (\exists \bar{x}\, (\delta(\bar{x}) \wedge \psi^{\mathcal{I}}))[\bar{a}_1, \ldots, \bar{a}_k] \\ &\iff \mathcal{M} \models (\exists x\, \psi)^{\mathcal{I}}[\bar{a}_1, \ldots, \bar{a}_k]. \end{aligned}$$

- $\theta = \forall x\, \psi$. This follows from the $\exists$ case via $\forall x\, \psi \equiv \neg\exists x\, \neg\psi$ and the $\neg$ case above. □

Remark 4.5. The proof demonstrates the importance of equivalence classes in our definition of interpretation. More specifically, notice that the surjectivity of $f_{\mathcal{I}}$ is used crucially in the existential case: every element of $N^{\mathcal{I}}$ *is* an equivalence class $\bar{c}\,/_{\varepsilon}$ for some $\bar{c} \in \delta(\mathcal{M}^n)$, so the existential quantifier over $N^{\mathcal{I}}$ translates well into an existential quantifier over $\delta(\mathcal{M}^n)$ in $\mathcal{M}$.

Theorem 4.6 (Interpretability transfers decidability)**.** *Let $\mathcal{M}$ and $\mathcal{N}$ be first-order structures. If $\mathcal{M}$ is decidable and $\mathcal{N} \cong \mathcal{N}^{\mathcal{I}}$ for some interpretation $\mathcal{I}$ of $\mathcal{N}$ in $\mathcal{M}$, then $\mathcal{N}$ is decidable.*

Proof. Let σ be any $\mathcal{L}_{\mathcal{N}}$-sentence. The translation map $\sigma \mapsto \sigma^{\mathcal{I}}$ is effective: it is defined recursively on the syntactic structure of σ, and each symbol of $\mathcal{L}_{\mathcal{N}}$ is assigned a fixed defining formula in $\mathcal{L}_{\mathcal{M}}$.

By the Truth Lemma (Lemma 4.4) and the isomorphism $\mathcal{N} \cong \mathcal{N}^{\mathcal{I}}$,

$$\mathcal{N} \models \sigma \iff \mathcal{N}^{\mathcal{I}} \models \sigma \iff \mathcal{M} \models \sigma^{\mathcal{I}}.$$

Since $\mathcal{M}$ is decidable there is an algorithm deciding $\mathcal{M} \models \sigma^{\mathcal{I}}$. The algorithm for $\mathcal{N}$ is therefore:

1. Compute $\sigma^{\mathcal{I}}$ recursively.
2. Apply the decision procedure for $\mathcal{M}$ to $\sigma^{\mathcal{I}}$.
3. Output the result.

Hence $\mathcal{N}$ is decidable. □

4.4 Interpreting Euclidean Geometry in RCF

We now show that Euclidean plane geometry is interpretable in RCF, completing the decidability proof.

The key idea is coordinate geometry: every point in the Euclidean plane $\mathbb{R}^2$ corresponds to a pair of real numbers. We use this to interpret the geometric primitives (betweenness and equidistance) as polynomial conditions on coordinates.

Theorem 4.7 (Geometry is interpretable in RCF)**.** *The theory* EG *of elementary Euclidean plane geometry is interpretable in* RCF*.*

Proof. We construct the (2 dimensional) interpretation $\mathcal{I}$ of the language of geometry $\mathcal{L}_{\text{geo}}$ in the language of ordered rings $\mathcal{L}_{\text{or}}$ explicitly. Let F be an arbitrary model of RCF.
1. Domain formula. Points in the Euclidean plane are represented by pairs of elements from the field F. We define the domain formula $\delta(x_1, x_2)$ to be $\top$, so that the set of coordinates is the entire Cartesian plane F^2.
2. Equality formula. Since the representation of points by coordinates is injective, two pairs (x_1, x_2) and (y_1, y_2) represent the same point if and only if they are identical. Thus, we set:

$$\varepsilon(x_1, x_2, y_1, y_2) \equiv (x_1 = y_1) \wedge (x_2 = y_2).$$

This formula trivially defines an equivalence relation on $\delta(F^2)$. The universe of our interpreted structure is $N^{\mathcal{I}} = F^2/_{\varepsilon} \cong F^2$.
3. Relation: Equidistance. The $\mathcal{L}_{\text{geo}}$-relation $xy \equiv zw$ holds if the Euclidean distances between the corresponding pairs are equal. We define $\varphi_{\equiv}(\bar{x}, \bar{y}, \bar{z}, \bar{w})$ as:

$$(x_1 - y_1)^2 + (x_2 - y_2)^2 = (z_1 - w_1)^2 + (z_2 - w_2)^2.$$

This is a quantifier-free $\mathcal{L}_{\text{or}}$-formula that is clearly compatible with ε.
4. Relation: Betweenness. The $\mathcal{L}_{\text{geo}}$-relation $\mathrm{B}(x, y, z)$ holds if y lies on the line segment xz. We define $\varphi_{\mathrm{B}}(\bar{x}, \bar{y}, \bar{z})$ as the conjunction of collinearity and the distance constraints:

$$\begin{aligned}&(y_1 - x_1)(z_2 - x_2) = (y_2 - x_2)(z_1 - x_1)\\ &\wedge\ (y_1 - x_1)^2 + (y_2 - x_2)^2 \leq (z_1 - x_1)^2 + (z_2 - x_2)^2\\ &\wedge\ (y_1 - z_1)^2 + (y_2 - z_2)^2 \leq (z_1 - x_1)^2 + (z_2 - x_2)^2.\end{aligned}$$

Since $a \leq b \equiv (a < b) \vee (a = b)$, $\leq$ is hence definable in RCF and this makes the relation above a valid $\mathcal{L}_{\text{or}}$-formula.

5. Translation map. The map $\mathcal{I}$ translates $\mathcal{L}_{\text{geo}}$-formulas to $\mathcal{L}_{\text{or}}$-formulas by relativizing quantifiers to δ. Because $\delta \equiv \top$, the translation simplifies: $(\exists x \phi)^{\mathcal{I}}$ becomes $\exists x_1 \exists x_2 \phi^{\mathcal{I}}$.

The axioms of EG (Tarski's axioms) are sent to sentences provable in RCF, as they correspond to polynomial properties of F^2. Thus, $\mathcal{I}$ is a valid interpretation. □

Corollary 4.8 (Decidability of Euclidean Geometry)**.** *The theory* EG *of elementary Euclidean plane geometry is decidable.*

Proof. By Theorem 3.19, RCF is decidable. By Theorem 4.7, EG is interpretable in RCF. By Theorem 4.6, EG is decidable. □

Corollary 4.9. *The theory of elementary n-dimensional Euclidean geometry is decidable for every $n \geq 1$.*

Proof sketch. Points in $\mathbb{R}^n$ are represented by n-tuples, and the interpretation of betweenness and equidistance is generalised to n coordinates. The same argument applies. □

5 Quantifier Elimination for the theory of cyclic abelian groups

In this section, we introduce a proof that the following structure admits quantifier elimination.

Definition 5.1 (Structure of Finite Abelian Group)**.** Let G be a finite abelian group, viewed as a first-order structure in the language

$$\mathcal{L} = \{0, +\}$$

Theorem 5.2. *If G is cyclic, then* $\mathrm{Th}(G)$ *admits quantifier elimination in the language $\mathcal{L}$.*

Proof. We define the structure in question to be $\mathcal{G} = (G, +, 0)$, interpreted in the language $\mathcal{L} = \{0, +\}$. We consider the following $\mathcal{L}$-theory axiomatized by:

(A) (Associativity):
$$\forall x \forall y \forall z \ ((x + y) + z = x + (y + z))$$

(B) (Identity):
$$\forall x \ (x + 0 = x)$$

(C) (Inverse):
$$\forall x \ \exists y \ (x + y = 0)$$

(D) (Commutativity):
$$\forall x \forall y \ (x + y = y + x)$$

(E) (Exponent):
$$\forall x \ (\overbrace{x + \cdots + x}^{n \text{ times}} = 0)$$

We call n the *order* of the group.

(F) (Generator):
$$\exists g \ \forall x \bigvee_{k=0}^{n-1} (x = \overbrace{g + \cdots + g}^{k \text{ times}})$$

Remark 5.3. Here are some remarks on our axiomatization.

1. Axioms $(A) - (D)$ describe the theory of abelian groups, and Axioms $(E) - (F)$ add cyclicity to the theory.
2. For simplicity, we will use the following notation:
$$n \cdot x := \overbrace{x + \cdots + x}^{n \text{ times}}$$
This will allow us to describe *multiplication* using addition, since multiplication is not defined in our language.

By Proposition 3.9, it suffices to eliminate $\exists x$ from a single conjunction of literals $\exists x \bigwedge_{i \in I} \phi_i(x, \bar{y})$, where each ϕ_i is a literal.

Since the only function and relation in the structure are $+$ and $=$ respectively, each literal $\phi_i(x, \bar{y})$ takes one of the following two forms:

$$\phi_i(x, \bar{y}) = \begin{cases} a_i \cdot x + p_i(\bar{y}) = 0 \\ \\ a_i \cdot x + p_i(\bar{y}) \neq 0 \end{cases}$$

where $0 \leq a_i \leq n$.

Now, if $a_i = 0$, then the formula $\phi_i(x, \bar{y}) \equiv p_i(\bar{y})$ is already quantifier-free and x does not appear, so we may remove it from the conjunction without issue.

Suppose then that $a_i \neq 0$. We can bring the $p_i(\bar{y})$ term to the other side of the equality using the existence of inverses in Axiom (C). Let $q_i(\bar{y})$ denote the inverse of $p_i(\bar{y})$, so we further reduce the forms that the literals $\phi_i(x, \bar{y})$ can take:

$$\phi_i(x, \bar{y}) = \begin{cases} a_i \cdot x = q_i(\bar{y}) \\ \\ a_i \cdot x \neq q_i(\bar{y}) \end{cases}$$

where $\big(0 \leq a_i \leq n \text{ and } a_i \neq 0\big) \Rightarrow 0 < a_i \leq n$.

So, now we are left to do quantifier elimination on the following arbitrary formula:

$$\exists x \ \phi(x, \bar{y}) \equiv \exists x \ \bigwedge_i \phi_i(x, \bar{y})$$

where each ϕ_i is of the form above.

To find a quantifier elimination method, we ask ourselves: *How do we find an x such that it satisfies every equality and inequality?*

First, we consider the equations. Say we have a conjunction of k equalities $\bigwedge_{i=1}^{k} a_i \cdot x = q_i(\bar{y})$. We want to find an x that satisfies all k equalities. It turns out that we can use the following theorem from number theory

Theorem 5.4 (Linear Congruence Theorem (LCT))**.** *Let a, b, n be integers with $n > 0$. The linear congruence*

$$ax \equiv b \pmod{n}$$

has a solution for x if and only if $d \mid b$, where $d = \gcd(a, n)$. If this condition is met, the solutions form a single residue class modulo $\frac{n}{d}$, namely:

$$x \equiv x_0 \pmod{\frac{n}{d}}$$

where x_0 is any particular solution.

Applying the Linear Congruence Theorem to each equation $a_i \cdot x = q_i(\bar{y})$ in the cyclic group G of order n, we see that a solution exists if and only if $\gcd(a_i, n) \mid q_i(\bar{y})$, and when it exists, the solutions form the residue class $x \equiv r_i(\bar{y}) \pmod{m_i}$ where $m_i := \frac{n}{\gcd(a_i, n)}$. So we have the following 2-way implication:

$$\exists x \ a_i \cdot x = q_i(\bar{y}) \iff \gcd(a_i, n) \mid q_i(\bar{y})$$

where n is the order of the cyclic group G as mentioned in Axiom (E).

With this fact, we have successfully reduced each equation to a quantifier free condition! But it is important for the reader to see that it is insufficient to say that this gives us quantifier elimination for a conjunction of equations. To illustrate this, we consider the following example.

Example 5.5. Consider the group $\mathbb{Z}/12\mathbb{Z}$ of order 12. Then, consider the conjunction of these 2 equations:

$$\Big(2 \cdot x = 4\Big) \wedge \Big(3 \cdot x = 3\Big)$$

Then, we are interested to find an x that satisfies *both* equations. By looking at both equations separately,

$$\begin{aligned}\exists x\ (2 \cdot x = 4) &\iff \gcd(2,12) = 2 \mid 4\ \checkmark \\ &\iff x \equiv 2 \pmod 6 \\ &\iff x \in \{2, 8\}\end{aligned}$$

$$\begin{aligned}\exists x\ (3 \cdot x = 3) &\iff \gcd(3,12) = 3 \mid 3\ \checkmark \\ &\iff x \equiv 1 \pmod 4 \\ &\iff x \in \{1, 5, 9\}\end{aligned}$$

Notice that even though individually, there exists an x that solves the equation, there is no x that solves both equations simultaneously.

This example aims to demonstrate that finding an x that satisfies each literal is insufficient. Rather, we need to find an x that satisfies every literal *simultaneously.*

But it turns out that this was shown to be possible in the proof of quantifier elimination for T_{Pres}! (Refer to 3.13) Specifically, after reducing each literal to a system of congruences (by LCT)

$$\begin{aligned}x &\equiv r_1(\bar{y}) \mod m_1 \\ x &\equiv r_2(\bar{y}) \mod m_2 \\ &\dots\end{aligned}$$

where each $m_i := \frac{n}{\gcd(a_i, n)}$, we can invoke the **Chinese Remainder Theorem** to get the following condition:

For every $i \neq j$,

$$r_i(\bar{y}) \equiv r_j(\bar{y}) \mod \gcd(m_i, m_j)$$

With this, we obtain a *quantifier-free* condition that will simultaneously satisfy every equation $\phi_i(x, \bar{y})$.

Now we have everything we need to get procedure to eliminate quantifiers from an arbitrary conjunction of literals. Given a conjunction of arbitrary literals:

$$\exists x\ \phi(x, \bar{y}) \equiv \exists x \bigwedge_i \phi_i(x, \bar{y})$$

We first look at the equations. After invoking the Linear Congruence Theorem and CRT on the literals, if the condition $r_i(\bar{y}) \equiv r_j(\bar{y}) \mod \gcd(m_i, m_j)$ is not satisfied, then the whole conjunction just returns false. Else, say it is satisfied, then it means that we have a nonempty *set* of values that x can take such that each equation is satisfied. We call this set the solution set, denoted by Δ. So,

$$\Delta := \left\{ x \in G : \bigwedge_{i \in I_{eq}} x \equiv r_i(\bar{y}) \mod m_i \right\}$$

Now, to deal with the inequations, recall that each inequation $a_j \cdot x \neq q_j(\bar{y})$ simply forbids x from taking a specific value in G. Concretely, the set of forbidden values is:

$$\mathcal{F} := \{x \in G : a_j \cdot x = q_j(\bar{y}) \text{ for some } j \in I_{\text{ineq}}\}$$

Since $|I_{\text{ineq}}|$ is finite, $\mathcal{F}$ is a finite set of at most $|I_{\text{ineq}}|$ elements. A solution to the full conjunction exists if and only if $\Delta \setminus \mathcal{F} \neq \emptyset$, that is, the solution set of the equations is not entirely exhausted by the forbidden values.

Since G is finite, this condition can be written explicitly as a finite disjunction:

$$\Delta \setminus \mathcal{F} \neq \emptyset \iff \bigvee_{g \in G} \left(\bigwedge_{i \in I_{\text{eq}}} g \equiv r_i(\bar{y}) \bmod m_i \ \wedge \bigwedge_{j \in I_{\text{ineq}}} a_j \cdot g \neq q_j(\bar{y}) \right)$$

where the disjunction ranges over all finitely many elements $g \in G$. Each term is a quantifier-free condition on $\bar{y}$, so the entire expression is quantifier-free.

And so, we have reduced

$$\exists x \bigwedge_{\ell} \phi_\ell(x, \bar{y}) \equiv \phi^*(\bar{y}) = \left(\Big(\bigwedge_{i,j} r_i(\bar{y}) \equiv r_j(\bar{y}) \mod \gcd(m_i, m_j) \Big) \wedge \Big(\Delta \setminus \mathcal{F} \neq \emptyset \Big) \right)$$

where the formula $\Delta \setminus \mathcal{F} \neq \emptyset$ is quantifier-free, as shown.

Finally, if the conjunction of literals are all inequations, that is if $I_{\text{eq}} = \emptyset$, then we set $\Delta = G$ (we do this because now we do not need a set of possible candidates of x, we can just choose from all $x \in G$), and the condition $\Delta \setminus \mathcal{F} \neq \emptyset$ reduces to $\bigvee_{g \in G} \bigwedge_{j \in I_{\text{ineq}}} a_j \cdot g \neq q_j(\bar{y})$, which is still a finite quantifier-free disjunction.

Therefore:

$$\exists x \bigwedge_{i} \phi_i(x, \bar{y}) \equiv \phi^*(\bar{y})$$

which is quantifier-free. By Proposition 3.9, T admits quantifier elimination. □

Example 5.6. We give an example of a finite abelian group G such that $\text{Th}(G)$ does not admit quantifier elimination in the language $\mathcal{L}$.

We use the following strategy: if $\text{Th}(G)$ admits QE, then for any formula $\phi(y)$ and any $a, b \in G$,

$$\big(G \models \psi(a) \iff G \models \psi(b) \ \text{ for all quantifier-free } \psi(y)\big) \implies \big(G \models \phi(a) \iff G \models \phi(b)\big).$$

Considering the contrapositive, it is sufficient to find $\phi(y)$ and $a, b \in G$ such that a and b is satisfied under the same quantifier-free formulas, yet ϕ distinguishes them.

Let $G = \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/4\mathbb{Z}$, and consider the elements

$$a = (0, 2), \qquad b = (1, 0).$$

Since quantifier-free formulas in one variable y are boolean combinations of atomic formulas $k \cdot y = 0$ (as defined in part 1), it suffices to check when $k \cdot y = 0$ holds. We compute:

$$k \cdot (0, 2) = (0, 2k \bmod 4) = (0, 0) \iff 2 \mid k,$$

$$k \cdot (1, 0) = (k \bmod 2,\ 0) = (0, 0) \iff 2 \mid k.$$

Hence a and b satisfy exactly the same quantifier-free formulas:

$$G \models \psi(a) \iff G \models \psi(b) \quad \text{for all quantifier-free } \psi(y).$$

Now consider the formula

$$\phi(y) :\equiv \exists x\, (x + x = y).$$

We claim $G \models \phi(a)$ but $G \not\models \phi(b)$.

Take $x = (0, 1)$. Since $(0, 1) + (0, 1) = (0, 2) = a$, it follows that $G \models \phi(0, 2)$.

However, for any $x = (m, n) \in G$, we have

$$x + x = (2m \bmod 2,\ 2n \bmod 4) = (0,\ 2n \bmod 4)$$

where its first component is always 0. Hence $x + x = (1, 0)$ has no solution.

Therefore $\phi(y)$ cannot be equivalent to any quantifier-free formula as any such formula supposedly should give the same truth value on a and b as explained above. But ϕ does not and so for this group, $\text{Th}(G)$ does not admit quantifier elimination in the language $\mathcal{L} = \{0, +\}$.

References

[1] David Marker. *An Invitation to Mathematical Logic*. Graduate Texts in Mathematics, vol. 301. Springer, 2024.

[2] Kurt Gödel. *Die Vollständigkeit der Axiome des logischen Funktionenkalküls*. Monatshefte für Mathematik und Physik, vol. 37, pp. 349–360, 1930.

[3] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012.

[4] Alfred Tarski. *A Decision Method for Elementary Algebra and Geometry*. RAND Corporation, 2nd edition, 1951.

[5] Wilfrid Hodges. *Model Theory*. Cambridge University Press, 2009.